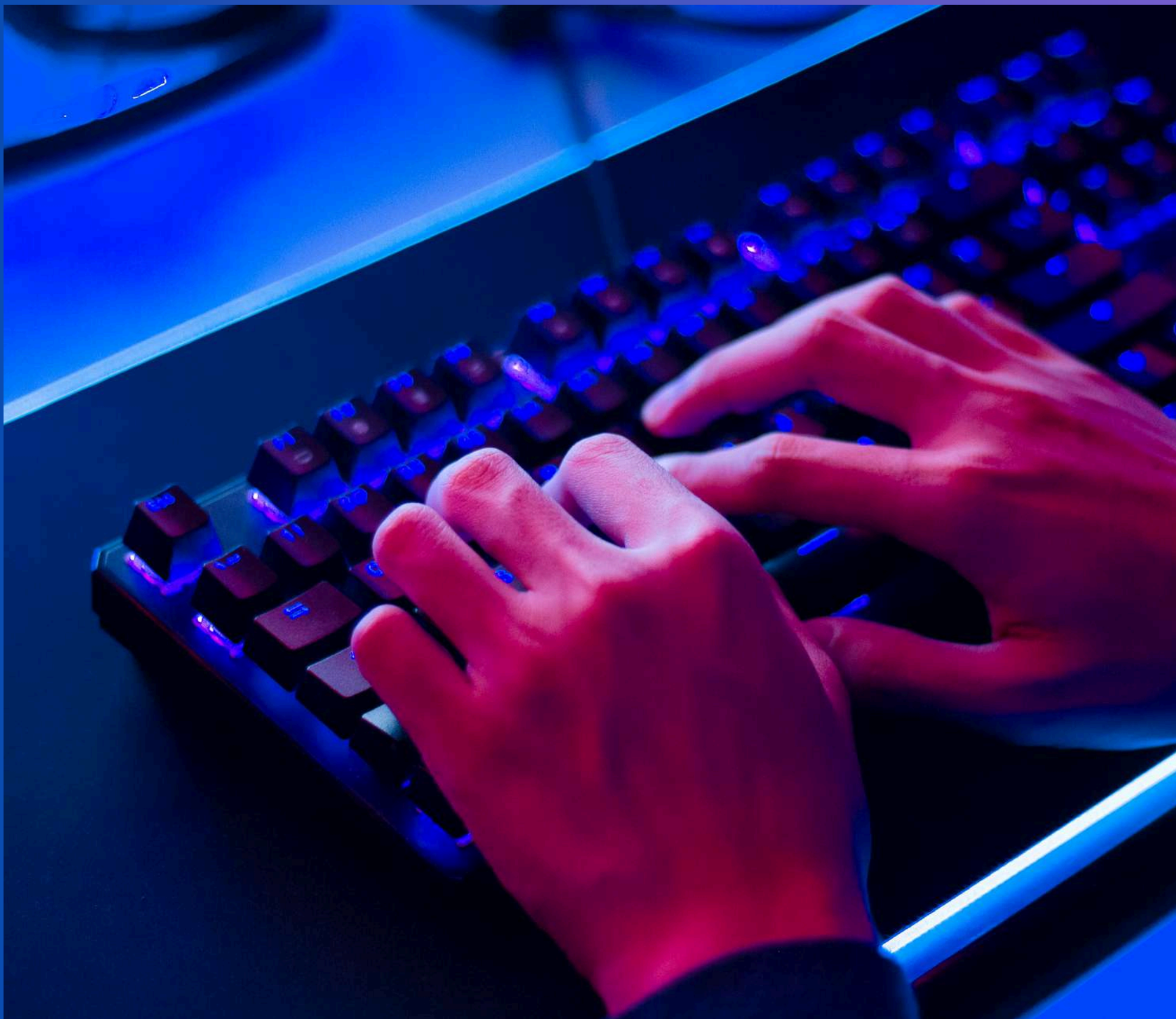




Tema 8: Motivación espacial y KD- Trees

OLIVER ANDRES QUIROZ MORA
CAROLINA GÓMEZ OSORNO
JUAN FELIPE MADROÑERO PAZOS





ÍNDICE



01 MOTIVACIÓN ESPECIAL

02 KD-TREE

03 BÚSQUEDA Y APLICACIONES



MOTIVACIÓN ESPACIAL

EL PROBLEMA ESPACIAL

Cuando los datos tienen coordenadas, las estructuras clásicas no escalan

EL PROBLEMA

Imagina millones de puntos en un mapa: ubicaciones GPS, píxeles, posiciones en videojuego.

Preguntas típicas:

- ▶ ¿Cuál restaurante queda más cerca?
- ▶ ¿Qué puntos caen en esta área?
- ▶ ¿Cuáles son los 5 vecinos más cercanos?

LA LIMITACIÓN

Con un array normal, la única opción es revisar TODOS los puntos uno por uno.

$O(n)$

por cada consulta

Con millones de datos y miles de consultas por segundo... esto no escala.

PROBLEMA 01 — VECINO MÁS CERCANO

Nearest Neighbor Search

Sin estructura: $O(n)$ → revisar todo
Con KD-Tree: $O(\log n)$ → saltar ramas

01

Definición

Dado un punto de consulta q , encontrar el punto más próximo en un conjunto S de puntos.

APLICACIONES REALES

GPS / Mapas

Uber buscando el conductor más cercano entre 10,000 activos en tiempo real

Reconocimiento facial

Comparar rostro con millones de vectores en base de datos

Diagnóstico médico

Clasificador kNN: pacientes similares → mismo diagnóstico

PROBLEMA 02 — BÚSQUEDA POR RANGO

Range Search

Sin estructura: $O(n)$

Con KD-Tree: $O(\sqrt{n} + k) \rightarrow k = \text{resultados}$

02

Definición

Encontrar TODOS los puntos que caen dentro de una región: rectángulo, círculo o cualquier forma.

APLICACIONES REALES



Google Maps

Mostrar todos los cafés en el área visible de la pantalla



Videojuegos

Detección de colisiones: objetos en el rango de la cámara



Astronomía

Catalogar todas las estrellas en un sector del cielo

PROBLEMA 03 — K VECINOS MÁS CERCANOS

k-Nearest Neighbors (k-NN)

Sin estructura: $O(n \log n)$

Con KD-Tree: $O(k \log n)$ → mucho más eficiente

03

Definición

Encontrar los k puntos más próximos a una consulta.
Generalización del vecino más cercano.

APLICACIONES REALES

 **Spotify / Netflix**

"Usuarios similares a ti también escucharon..."

 **Machine Learning**

Algoritmo kNN: clasificar nuevos puntos por vecindad

 **Computer Vision**

Reconocimiento de objetos con descriptores SIFT

¿POR QUÉ NO SIRVEN ESTRUCTURAS 1D?

Ninguna estructura unidimensional captura la geometría del espacio multidimensional

ESTRUCTURA	DATOS 1D	DATOS 2D+	RAZÓN
BST / Árbol binario	✓ Perfecto	✗ No aplica	Solo ordena en 1 dimensión
Array ordenado	✓ Búsq. binaria	✗ ¿Por cuál eje?	Ambiguo en multi-dim
Hash Table	✓ $O(1)$ exacto	✗ Sin proximidad	No hay noción de cercanía
KD-Tree	✓ OK	✓ Generaliza k-D	Divide el espacio alternando ejes

La "maldición de la dimensionalidad": en espacios de muchas dimensiones, los índices 1D pierden toda utilidad

COMPARACIÓN DE COMPLEJIDADES

Fuerza bruta vs. KD-Tree — la diferencia es dramática a escala

OPERACIÓN	FUERZA BRUTA	KD-TREE (prom.)	MEJORA
Construcción	$O(n)$	$O(n \log n)$	
Nearest Neighbor	$O(n)$	$O(\log n)$	⚡ mucho mejor
Range Search	$O(n)$	$O(\sqrt{n} + k)$	⚡ mucho mejor
k-NN	$O(n \log n)$	$O(k \log n)$	⚡ mucho mejor

n = número de puntos | k = número de resultados | Complejidad espacial: $O(n)$ en todos los casos

LA SOLUCIÓN: DIVIDIR EL ESPACIO

¿Y si dividimos el espacio recursivamente, alternando dimensiones?

1

Nivel 1: Dividir por X

Separar puntos según coordenada X. Los puntos menores van a la izquierda, mayores a la derecha.



2

Nivel 2: Dividir por Y

En cada mitad, separar por coordenada Y. Ahora tenemos 4 cuadrantes.



3

Nivel 3: Dividir por X...

El patrón se repite recursivamente. El eje alterna: $X \rightarrow Y \rightarrow X \rightarrow Y \rightarrow \dots$

↳ Esto es exactamente lo que hace un KD-TREE: un BST que particiona el espacio k-dimensional de forma recursiva

APLICACIONES DEL MUNDO REAL

Dónde usamos búsquedas espaciales eficientes todos los días

Google Maps / Waze

Puntos de interés, rutas y conductores más cercanos en tiempo real

Spotify / Netflix

Recomendaciones: similitud en espacio de características (k-NN)

Videojuegos

Detección de colisiones y carga de entidades en rango de cámara

Robótica (SLAM)

Mapeo y navegación autónoma usando nubes de puntos 3D

Astronomía

Catálogo de estrellas y búsqueda de objetos celestes en sector

Computer Vision

Reconocimiento de objetos con descriptores SIFT / ORB

CONCLUSIÓN

- Los datos espaciales (2D, 3D, nD) son comunes: GPS, visión, ML, física...
- Las estructuras clásicas (arrays, BSTs, hash tables) no escalan $\rightarrow O(n)$ por consulta
- Necesitamos búsqueda eficiente de vecinos y rangos en múltiples dimensiones

↳ La solución:

dividir el espacio recursivamente, alternando ejes en cada nivel.

Esto es exactamente lo que define al KD-TREE $\rightarrow$



ÁRBOL BINARIO

Ordena por una sola clave

KD-TREE

Divide el espacio con un hiperplano, alternando el eje ($x \rightarrow y \rightarrow z \rightarrow \dots$)

01



02



KD-TREE

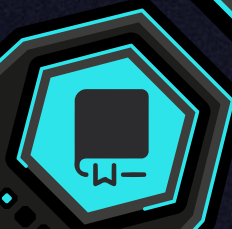
Almacena vectores

KD-TREE

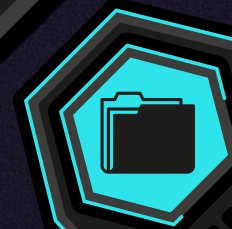
Se construye balanceado para ser eficiente

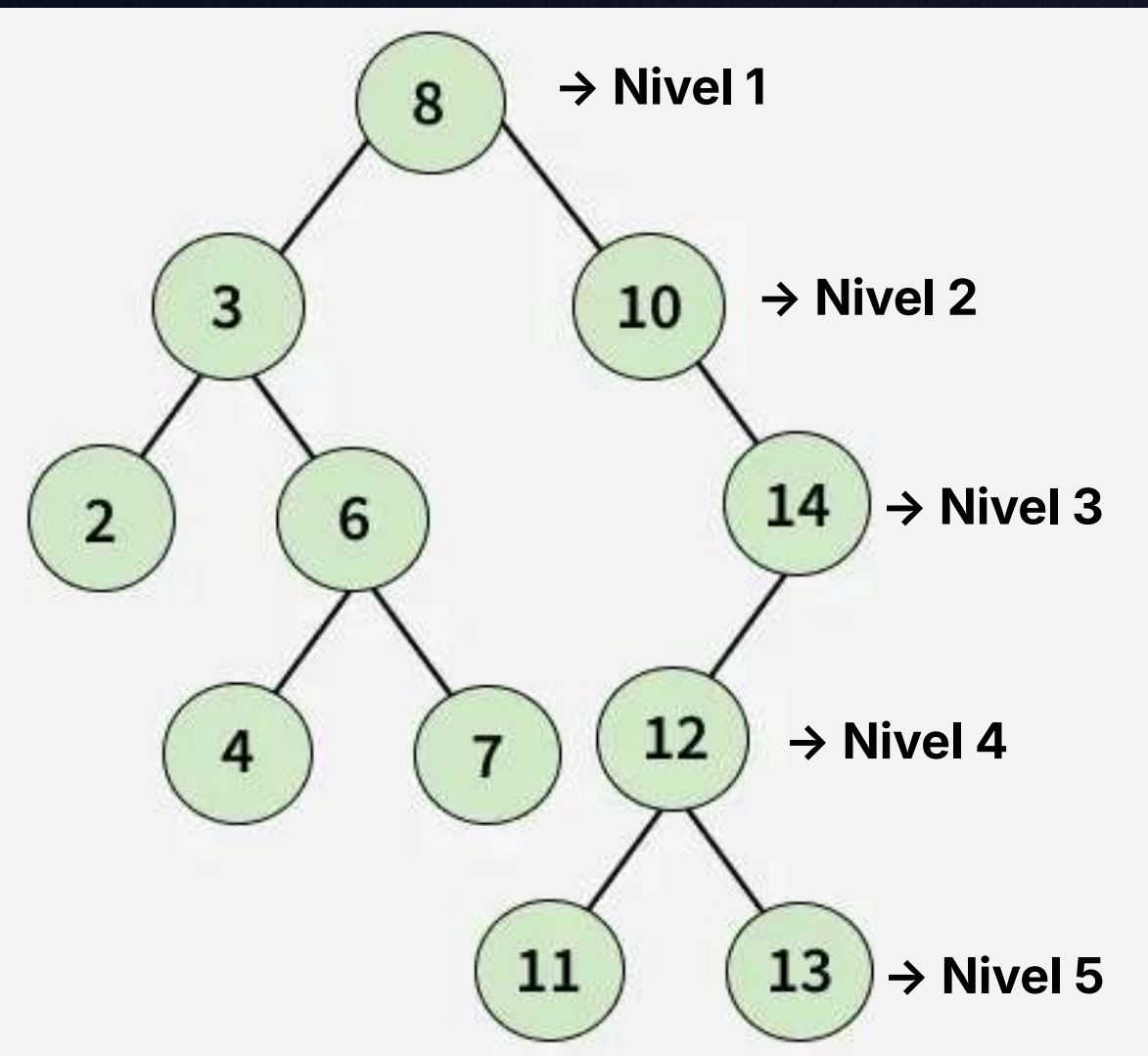
Diferencia entre
KD-Tree y Árbol
Binario

03

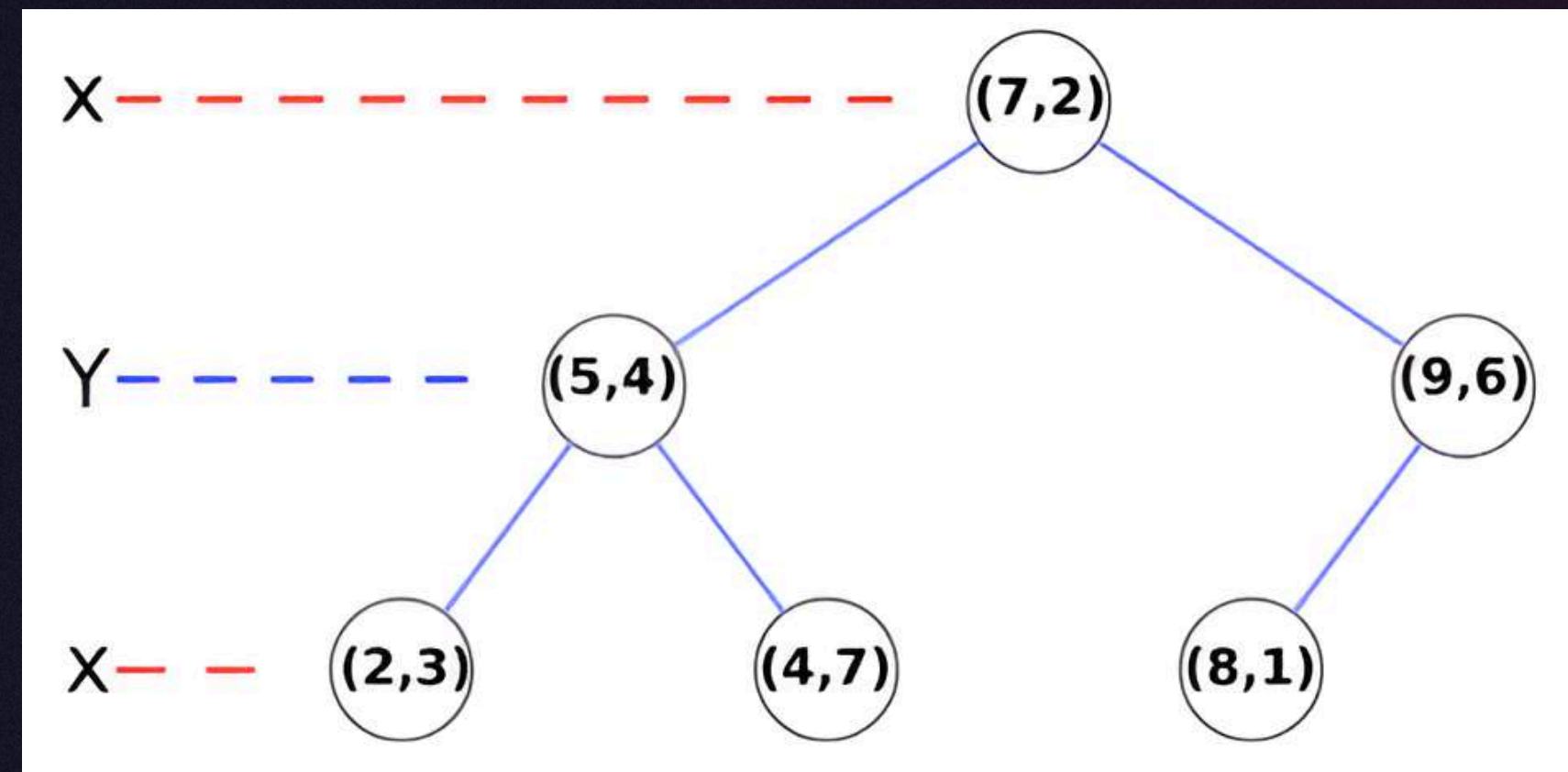


04





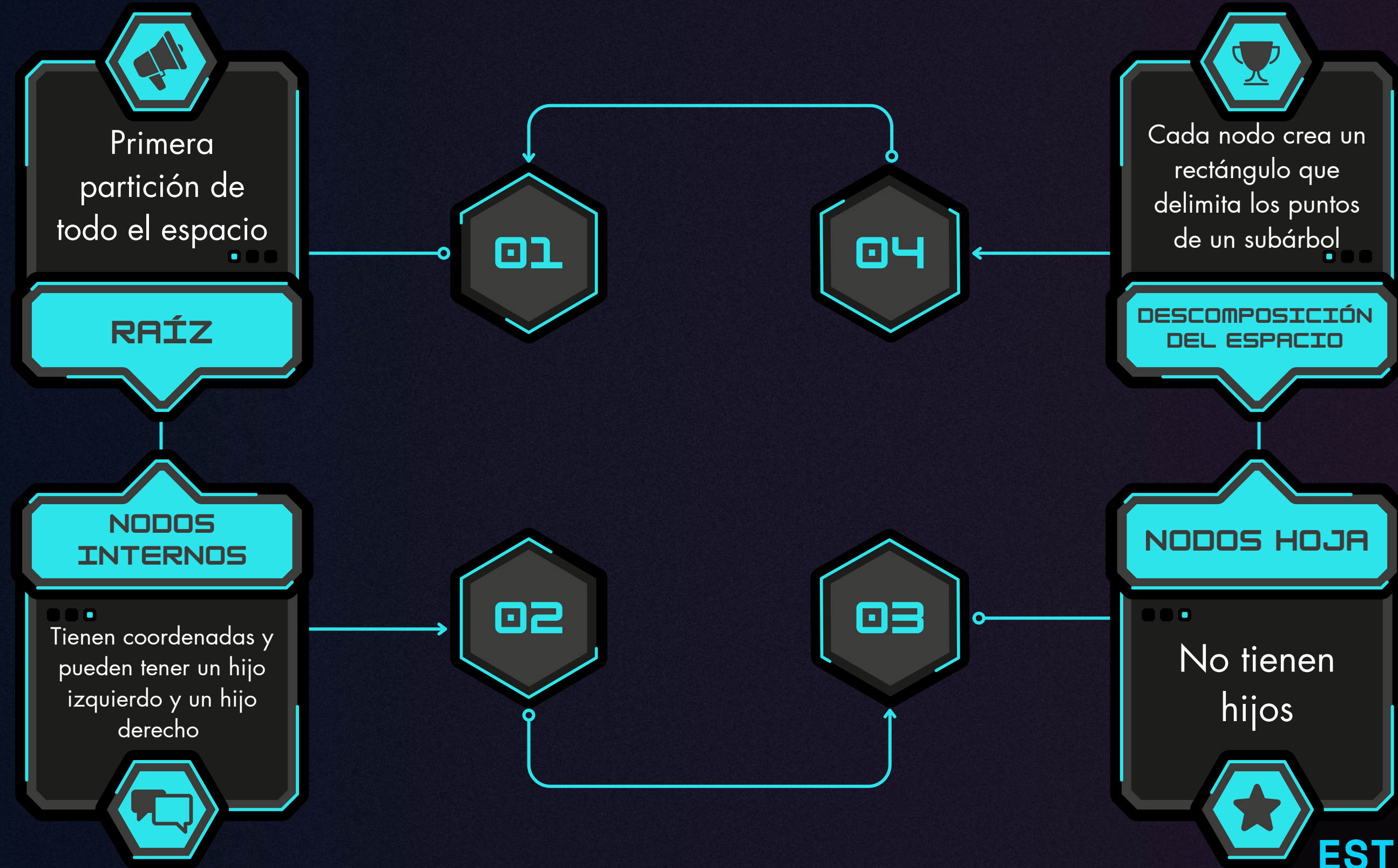
ÁRBOL BINARIO



KD-TREE



CARACTERÍSTICAS



ESTRUCTURA

OPERACIÓN	COMPLEJIDAD TEMPORAL	COMPLEJIDAD ESPACIAL
INSERTAR	• Promedio de $O(\log n)$ • Peor de $O(n)$	• Promedio de $O(\log n)$ • Peor de $O(n)$
ELIMINAR	• Promedio de $O(\log n)$ • Peor de $O(n)$	• Promedio de $O(\log n)$ • Peor de $O(n)$
BUSCAR	---	---

OPERACIONES

PRIMER ELEMENTO

Se quiere insertar el punto [5,4]

SEGUNDO ELEMENTO

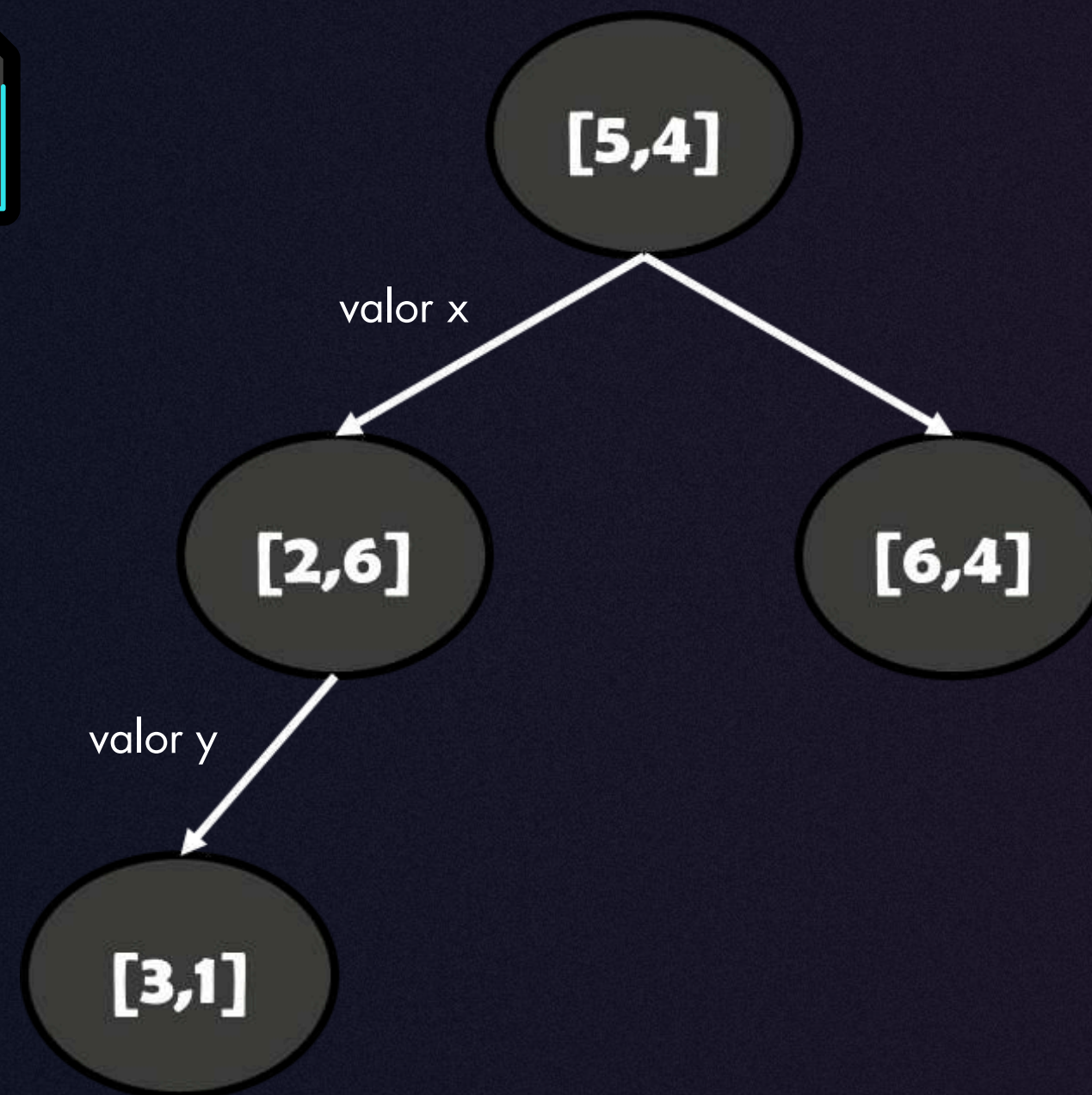
Se quiere insertar el punto [2,6]

TERCER ELEMENTO

Se quiere insertar el punto [3,1]

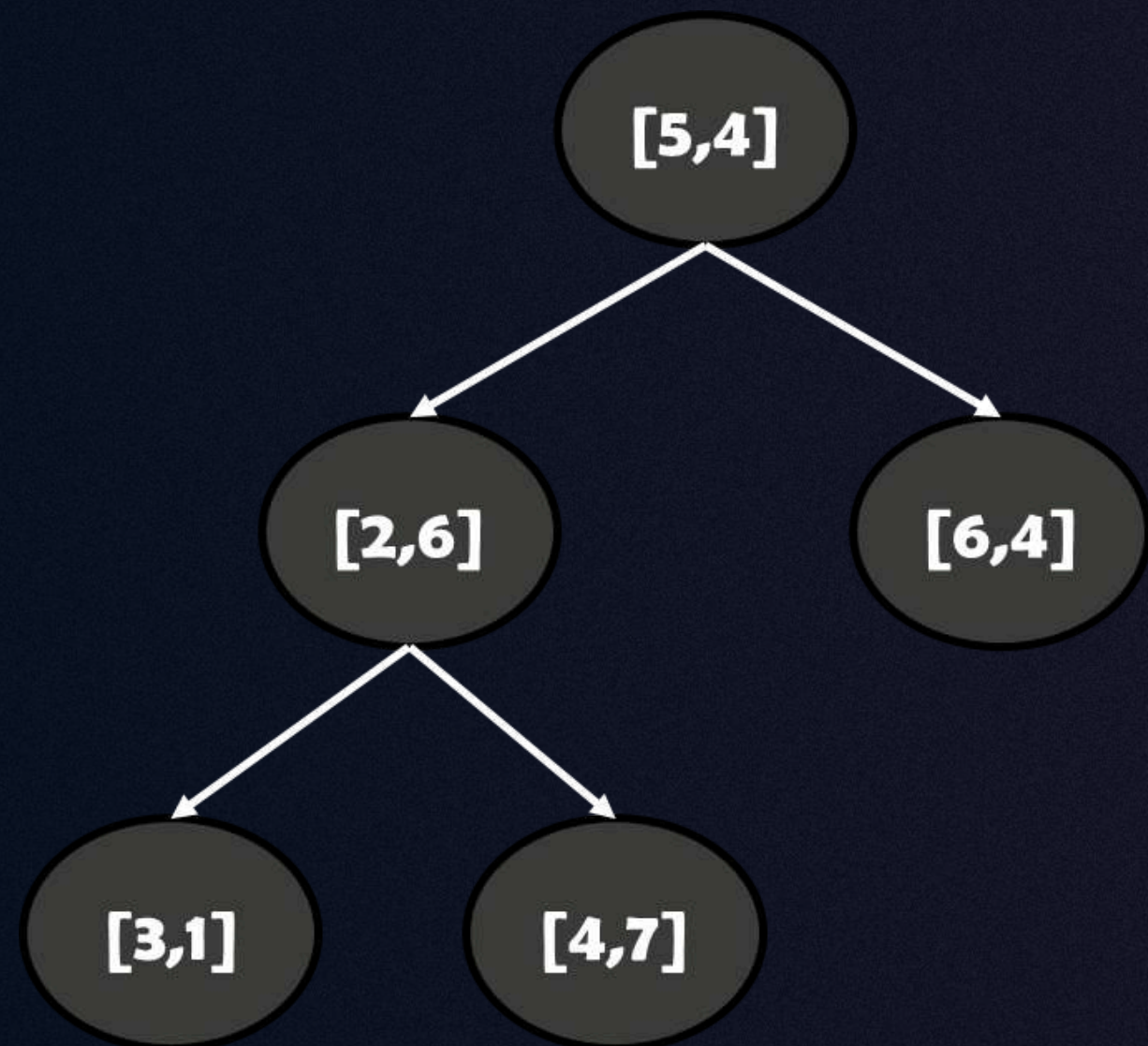
CUARTO ELEMENTO

Se quiere insertar el punto [6,4]



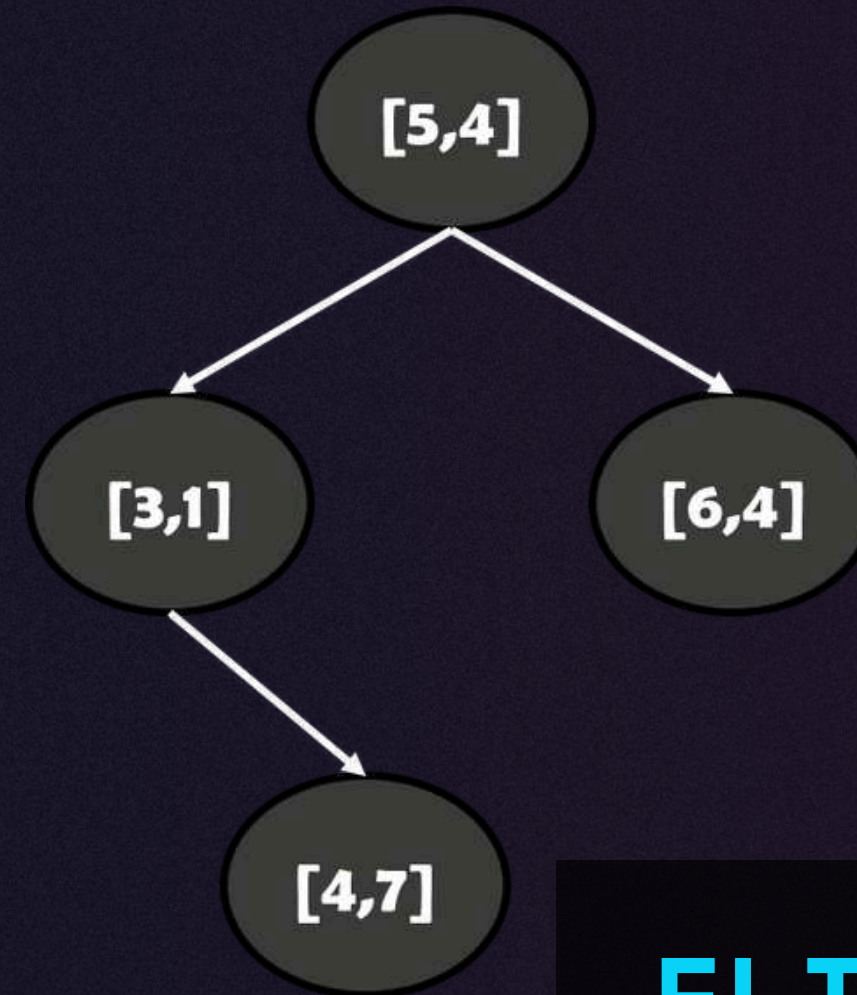
INSERCIÓN

Se quiere eliminar el punto [2,6] del siguiente árbol:



PROCESO

Si tiene descendientes, buscar el valor mínimo correspondiente al eje, reemplazar y eliminar



ELIMINACIÓN

CONSTRUCCIÓN

La inserción de cada nodo sigue un recorrido desde la raíz alternando el eje de comparación en cada nivel.

Insertar [2,3]

[2,3]

Insertar [4,7]

[2,3]

$4 > 2$

[4,7]

Insertar [5,4]

[2,3]

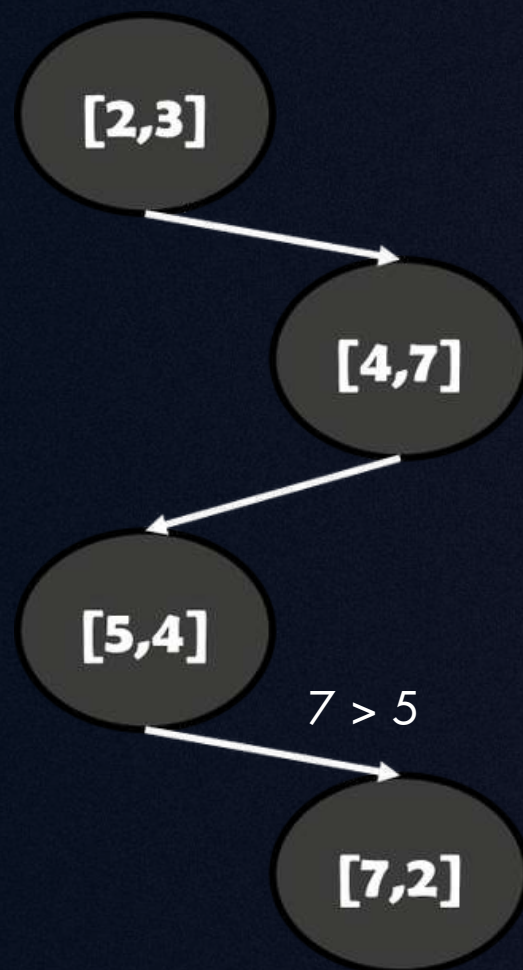
[4,7]

$4 < 7$

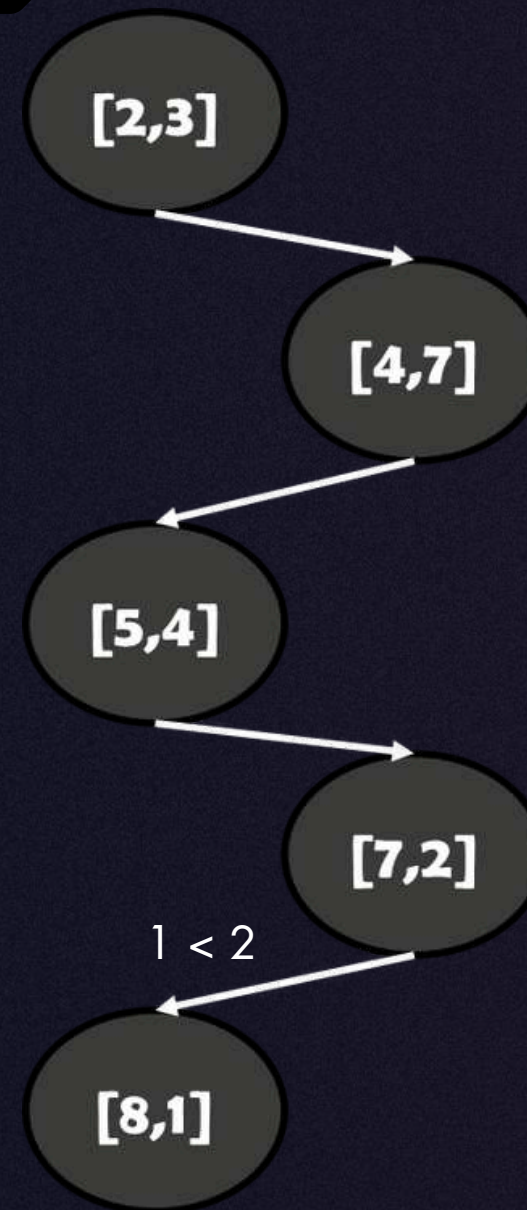
[5,4]

ÁRBOL DESBALANCEADO

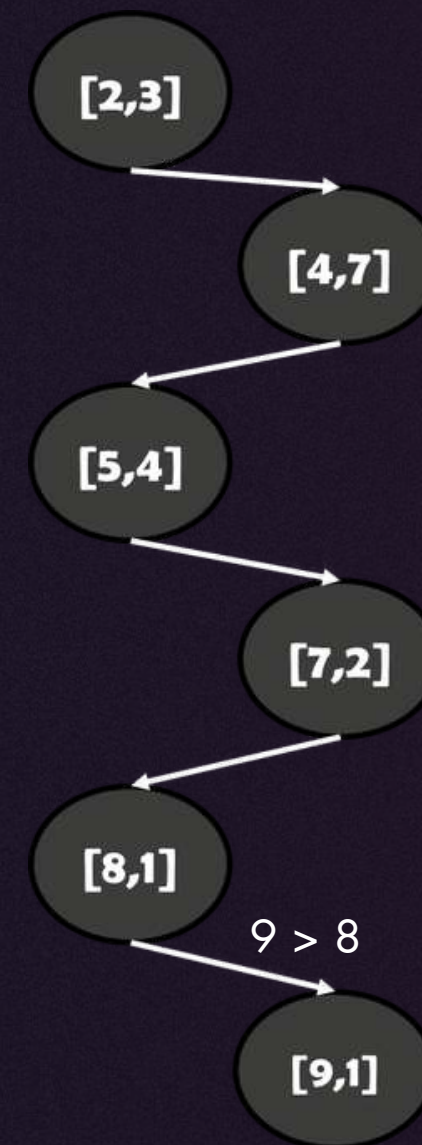
Insertar [7,2]



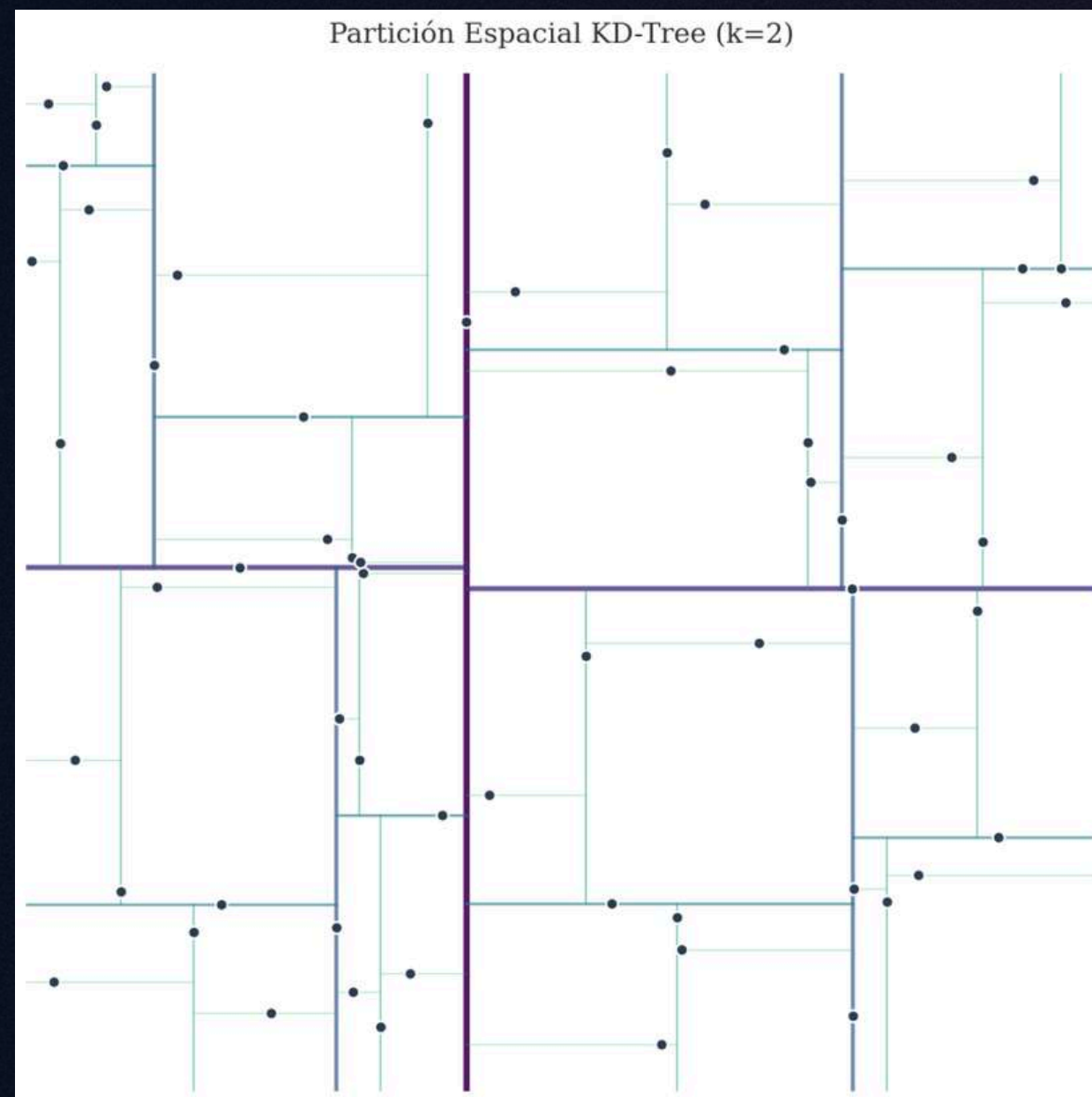
Insertar [8,1]



Insertar [9,1]



ÁRBOL DESBALANCEADO



POR NIVEL

01

Cada nivel = un
hiperplano ortogonal
a un eje

ALTERNANCIA

02

Se alterna
sistemáticamente entre
dimensiones

PARTICIÓN DEL ESPACIO

CÁLCULO EJE

01

Eje = profundidad mod k

SELECCIÓN MEDIANA

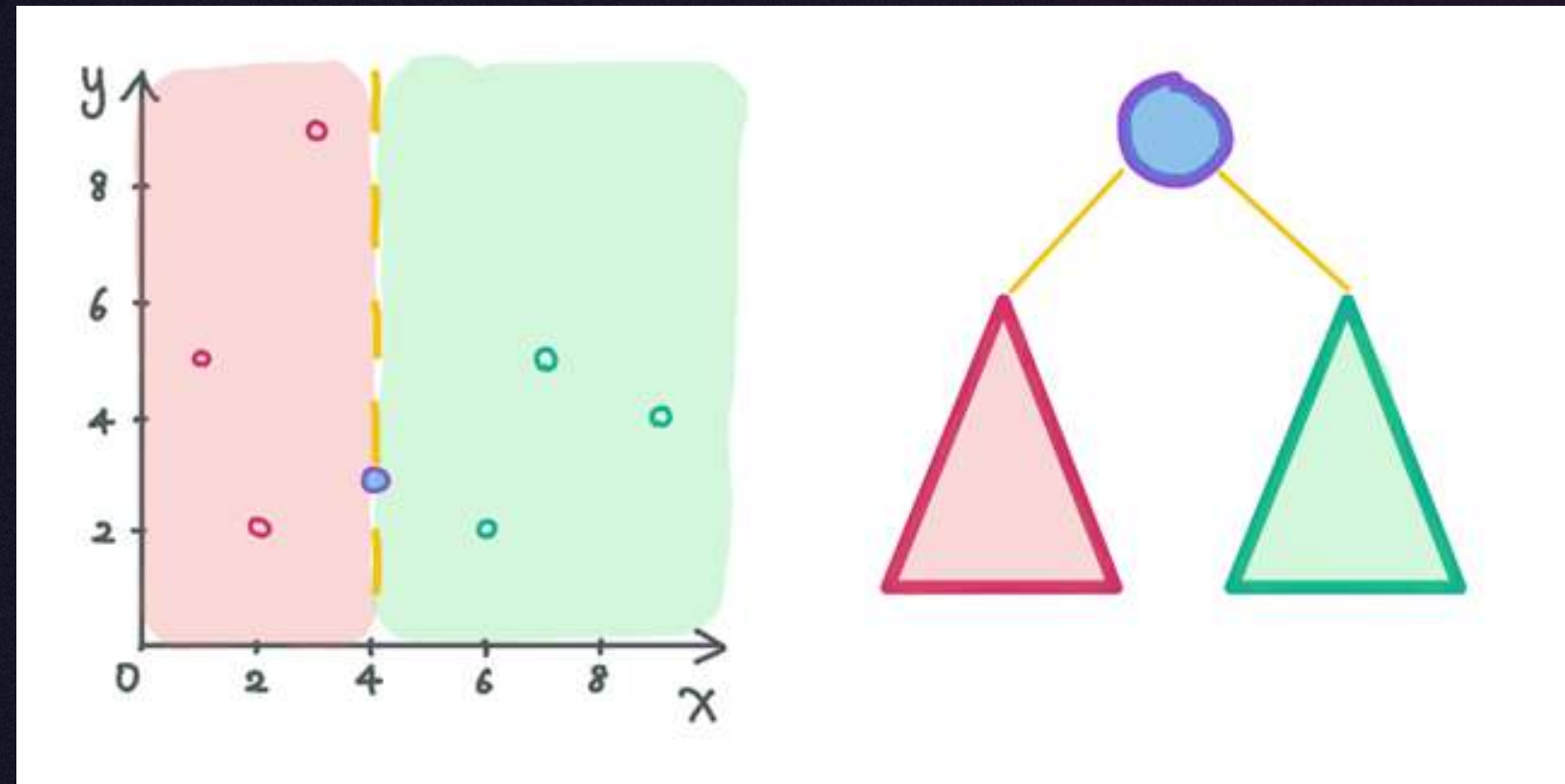
02

Se ordenan los puntos por el eje calculado y se elige la mediana como nodo

RECURSIVIDAD

03

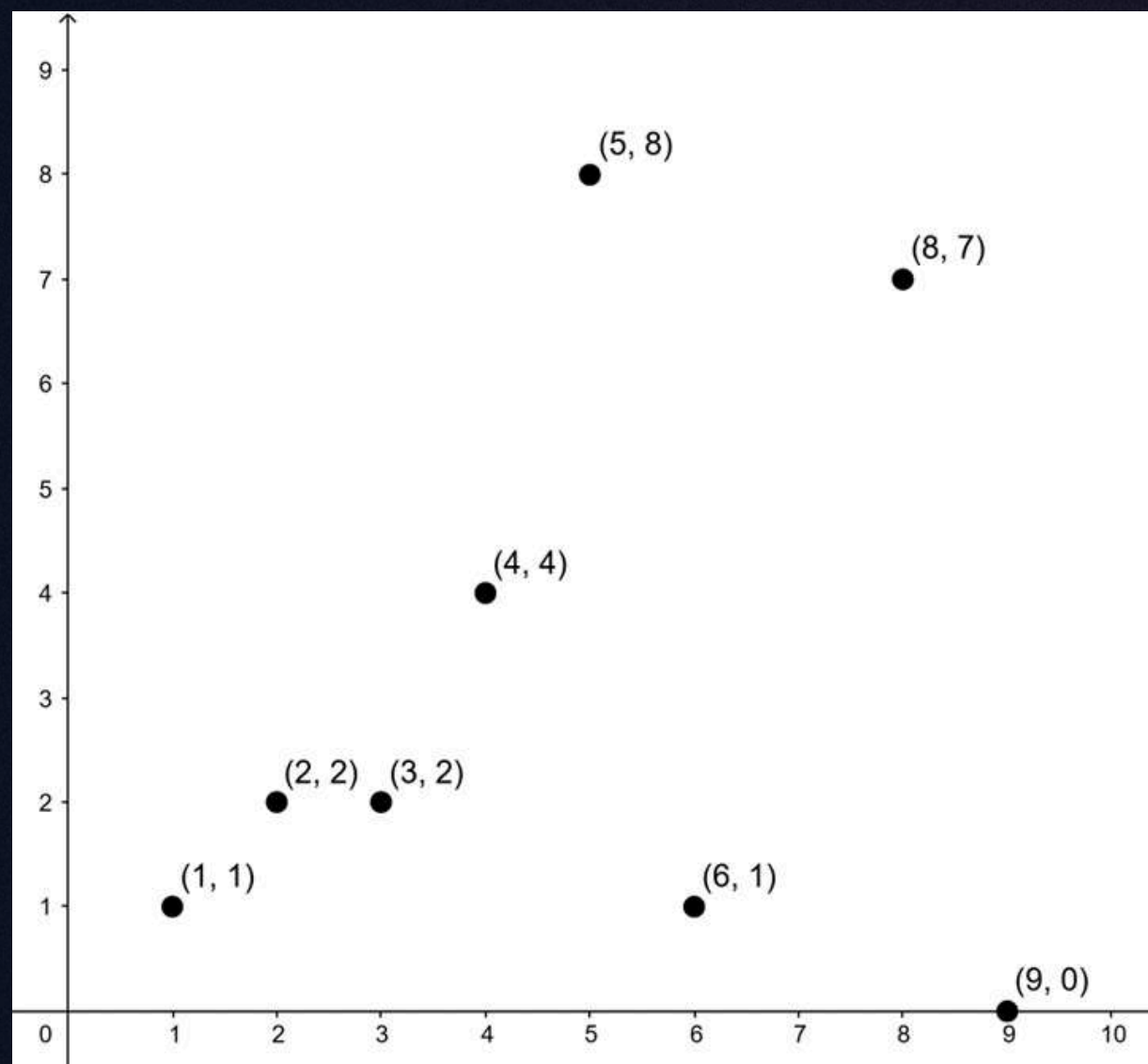
El proceso se repite para cada subconjunto de datos



SELECCIÓN DE LA MEDIANA

✕ Eje = 0 mod 2 = 0

(3, 2)	(5, 8)	(6, 1)	(9, 0)	(4, 4)	(1, 1)	(2, 2)	(8, 7)
--------	--------	--------	--------	--------	--------	--------	--------



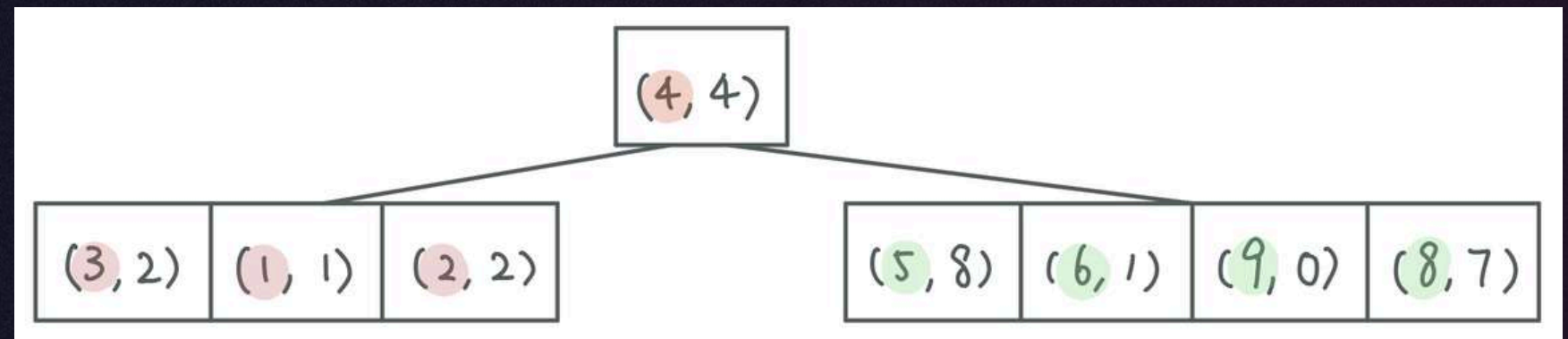
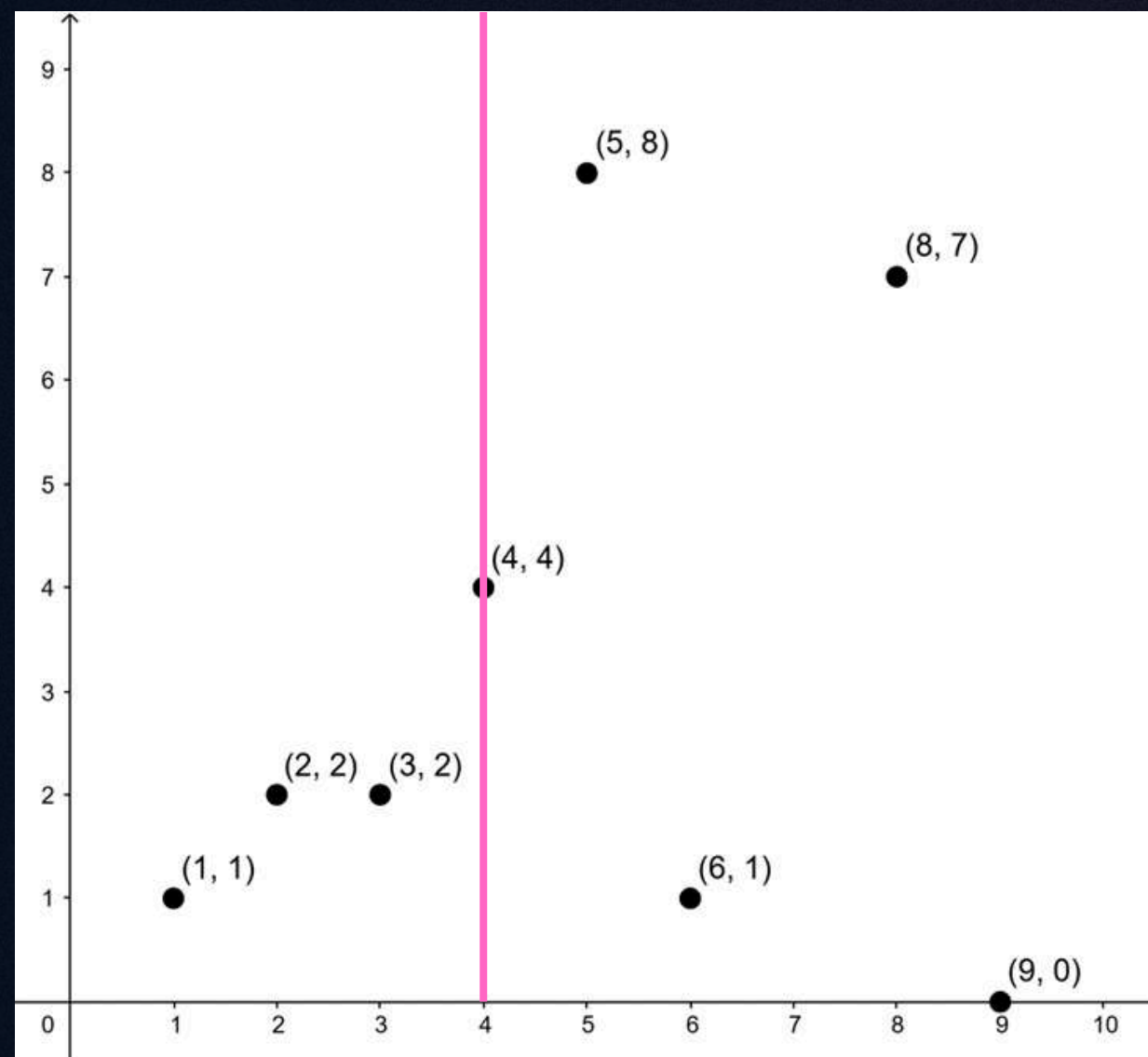
PUNTOS ORDENADOS

(1, 1), (2, 2), (3, 2),
(4, 4), (5, 8), (6, 1),
(8, 7), (9, 0)

● ● ●

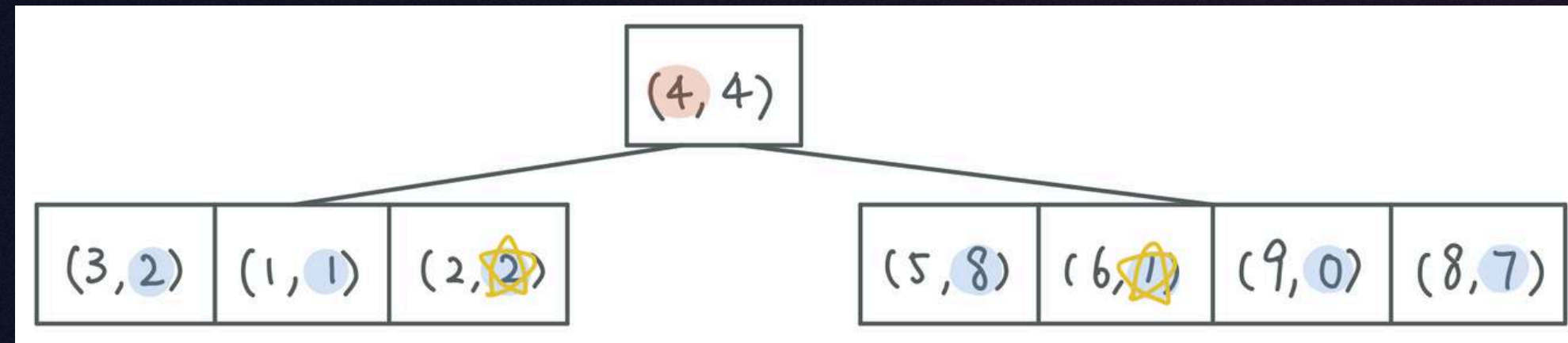
SELECCIÓN DE LA MEDIANA

$(3, 2)$	$(5, 8)$	$(6, 1)$	$(9, 0)$	$(4, 4)$	$(1, 1)$	$(2, 2)$	$(8, 7)$
----------	----------	----------	----------	----------	----------	----------	----------



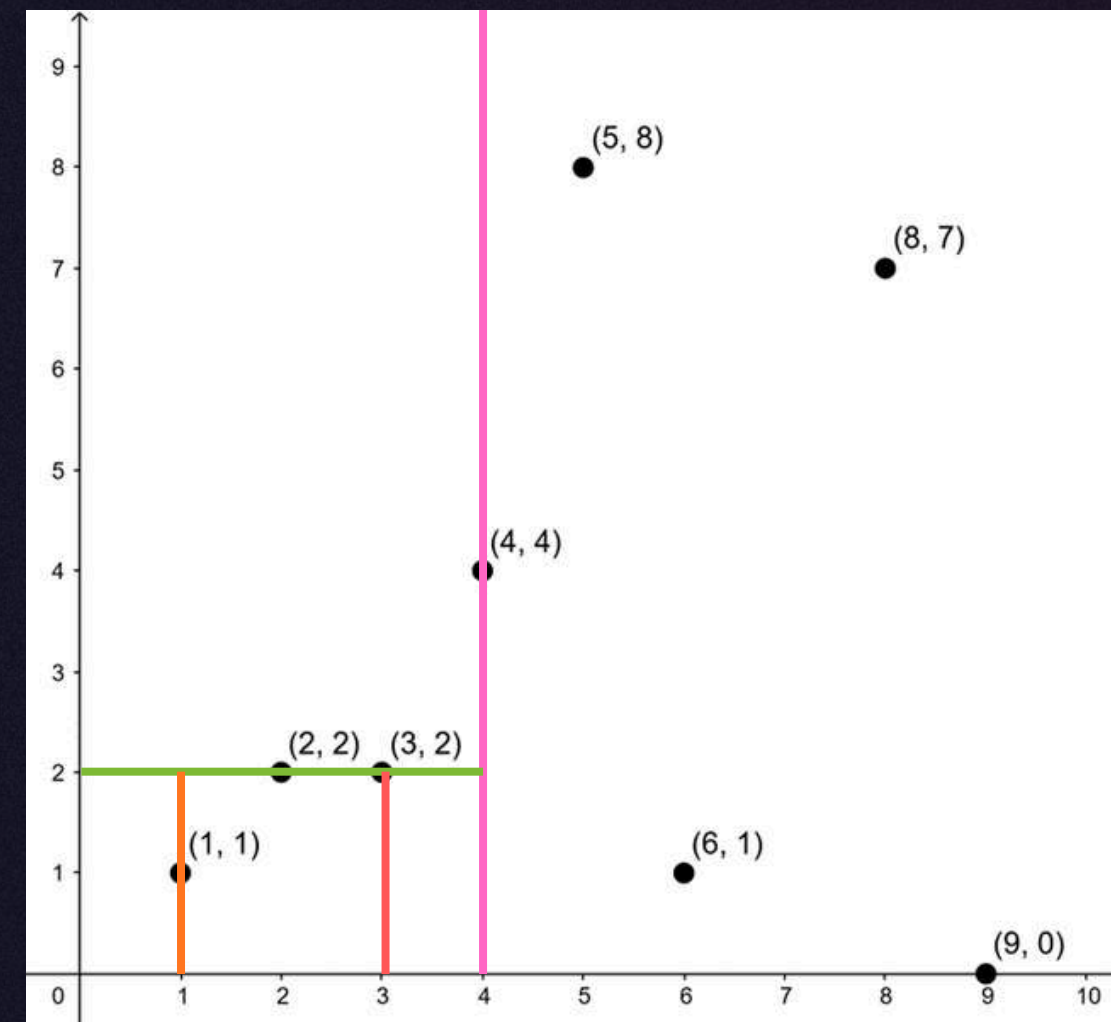
SELECCIÓN DE LA MEDIANA

$Eje = 1 \bmod 2 = 1$



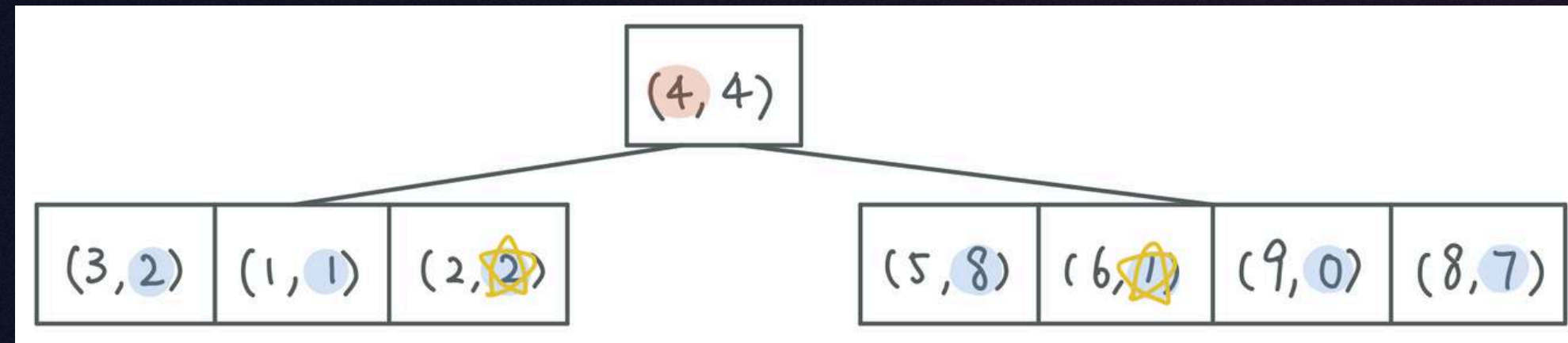
PUNTOS ORDENADOS

$(1, 1), (2, 2), (3, 2)$

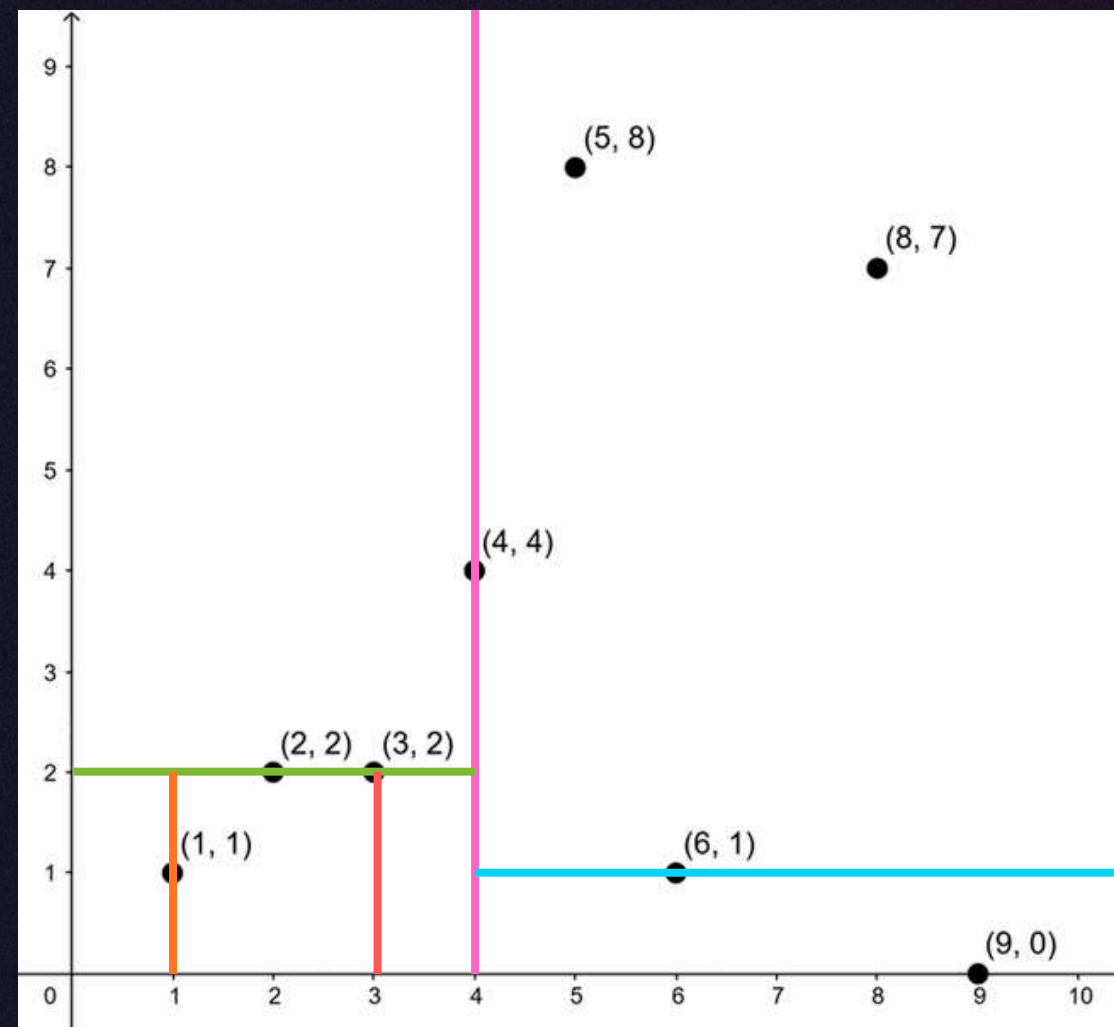


SELECCIÓN DE LA MEDIANA

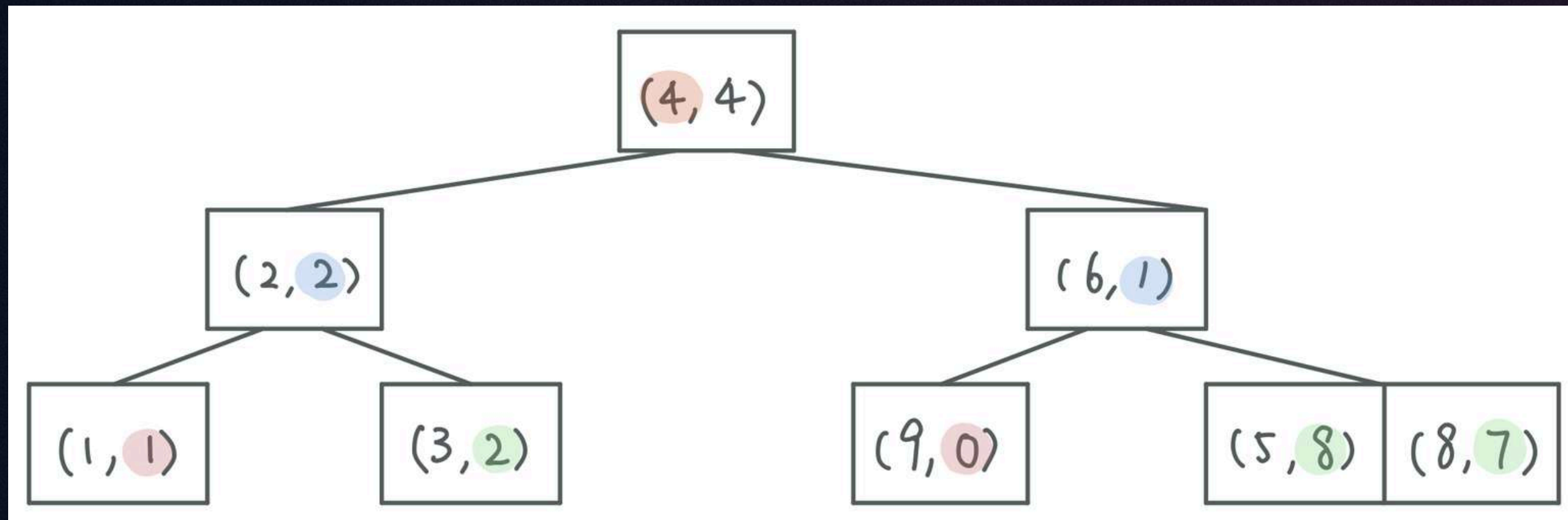
$Eje = 1 \bmod 2 = 1$



PUNTOS ORDENADOS
 $(9, 0)$, $(6, 1)$, $(8, 7)$,
 $(5, 8)$



SELECCIÓN DE LA MEDIANA

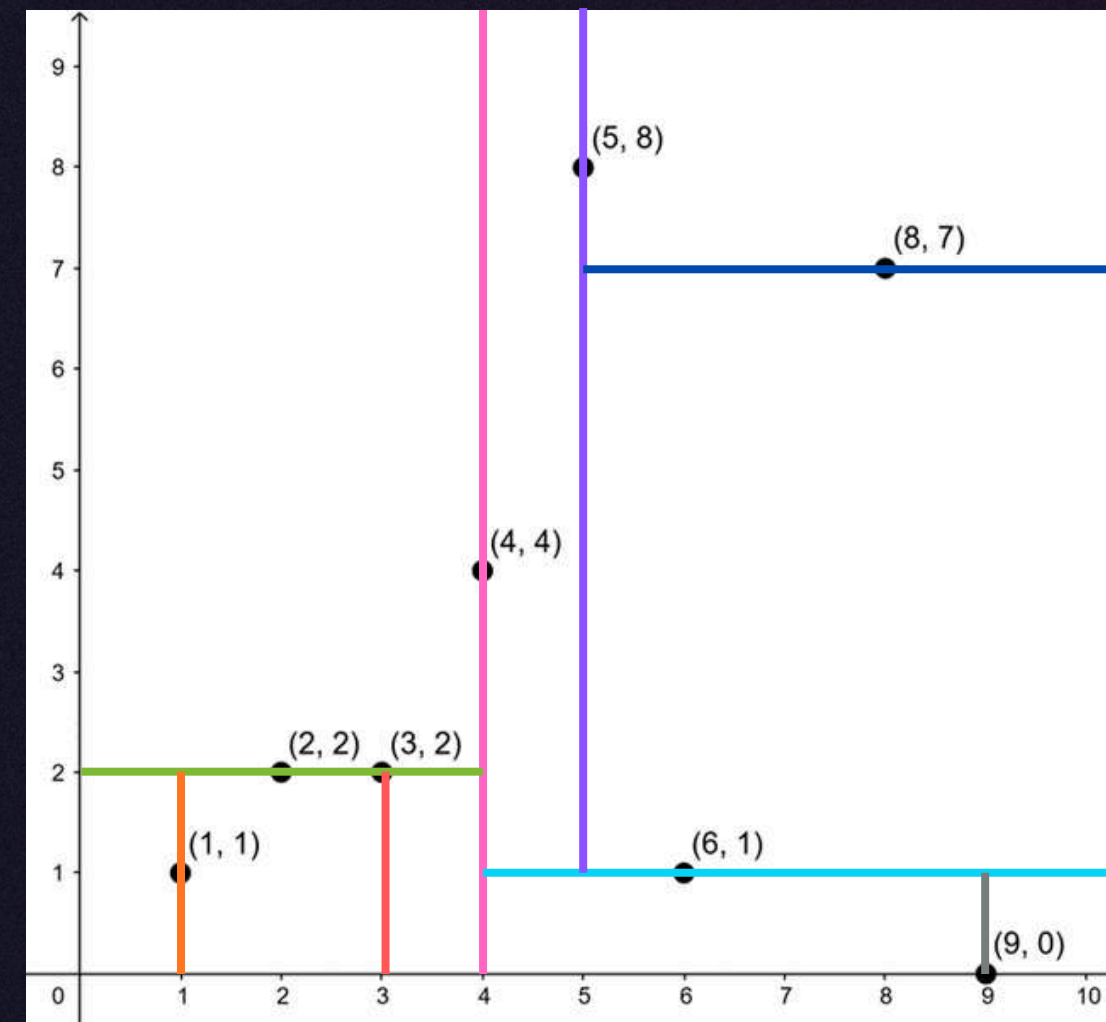
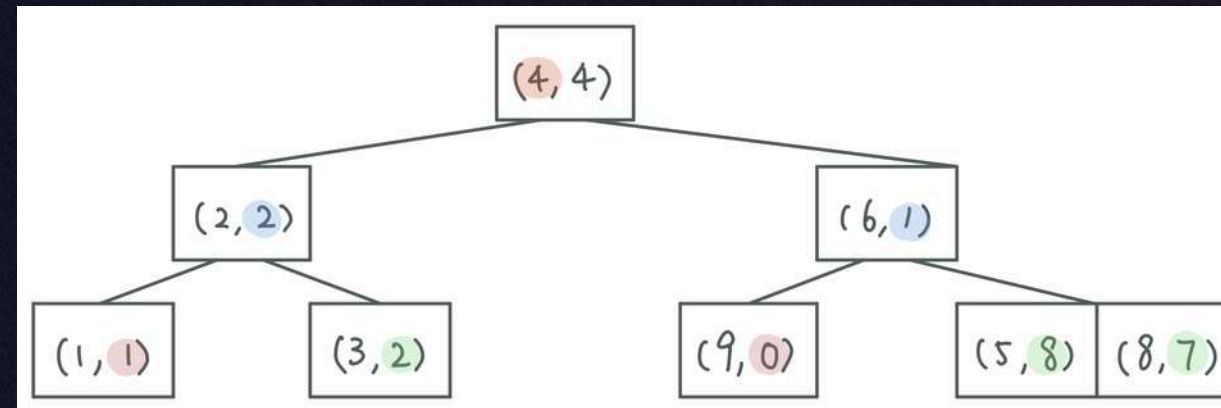


SELECCIÓN DE LA MEDIANA

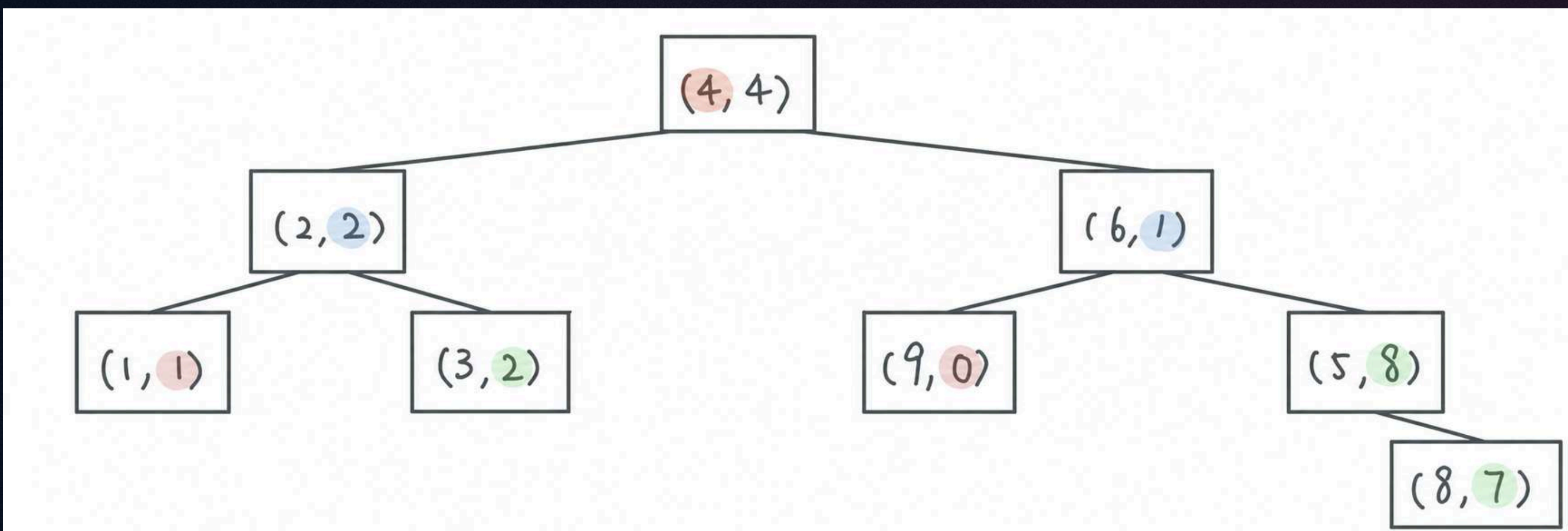
X Eje = $2 \bmod 2 = 0$

PUNTOS ORDENADOS

(5,8), (8,7)



SELECCIÓN DE LA MEDIANA



SELECCIÓN DE LA MEDIANA



PASOS PARA SU CONSTRUCCIÓN

CASO PROMEDIO

$$O(kn \log n)$$



COMPLEJIDAD DE
CONSTRUCCIÓN

PEOR CASO

$$O(kn^2)$$

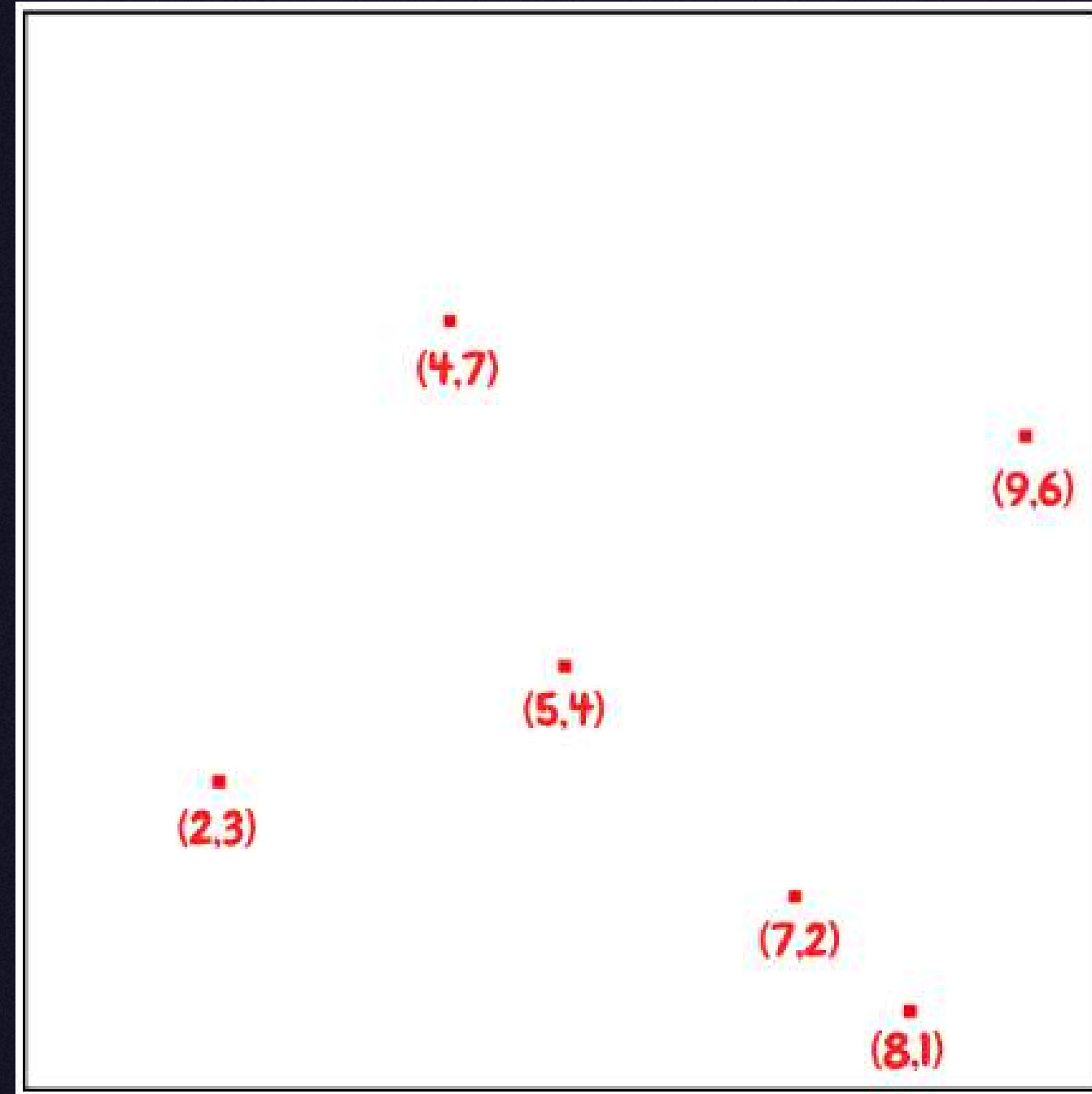


CONSTRUCCIÓN - COMPLEJIDAD

ENUNCIADO

Construir un KD-Tree con
los siguientes puntos:
(2,3), (5,4), (9,6), (4,7),
(8,1), (7,2)

...



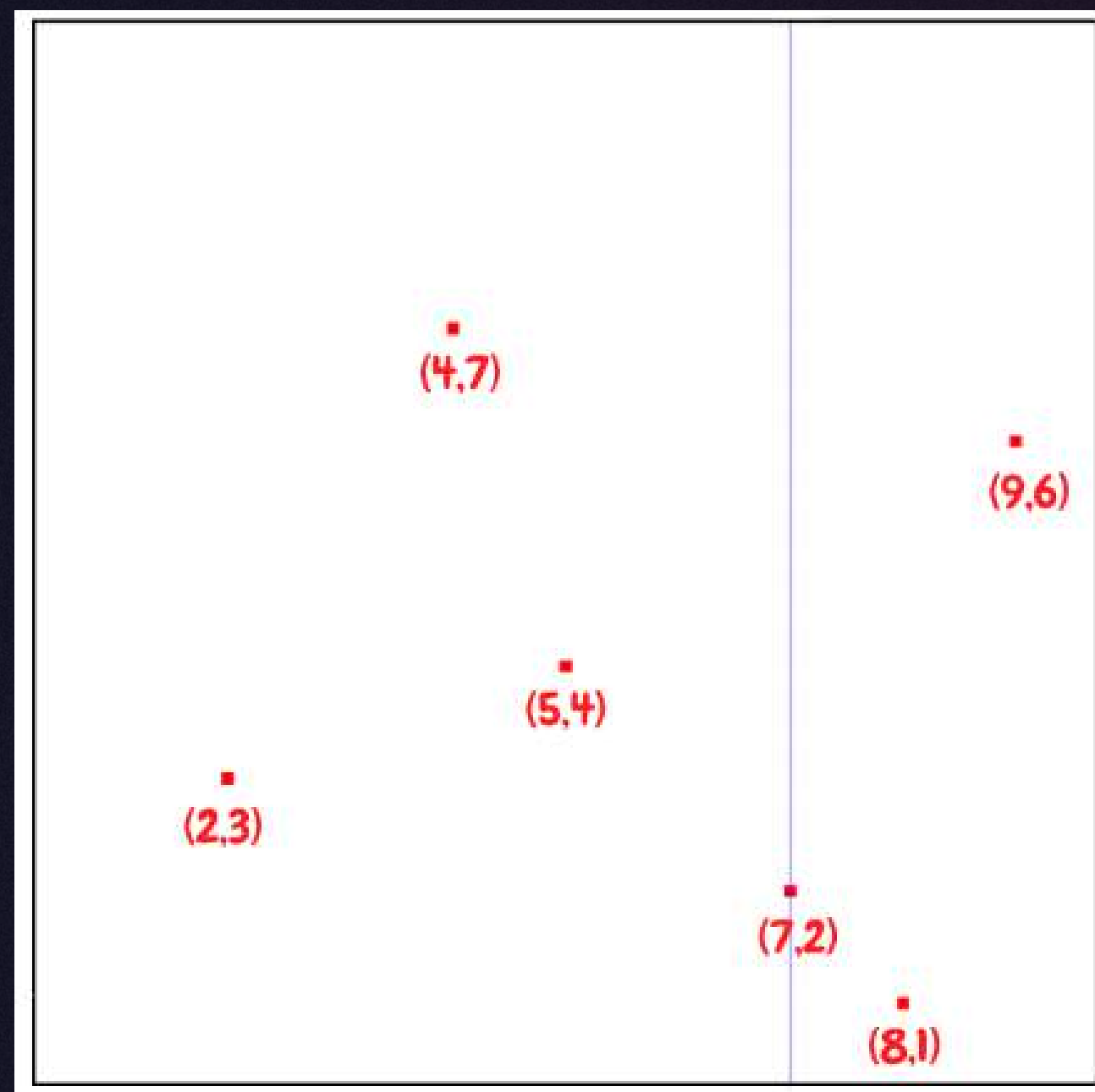
EJEMPLO EN 2D

(2, 3), (4, 7), (5, 4), (7, 2),
(8, 1), (9, 6)

× Eje = $0 \bmod 2 = 0$

PUNTOS ORDENADOS

(2, 3), (4, 7), (5, 4),
(7, 2), (8, 1), (9, 6)



EJEMPLO EN 2D

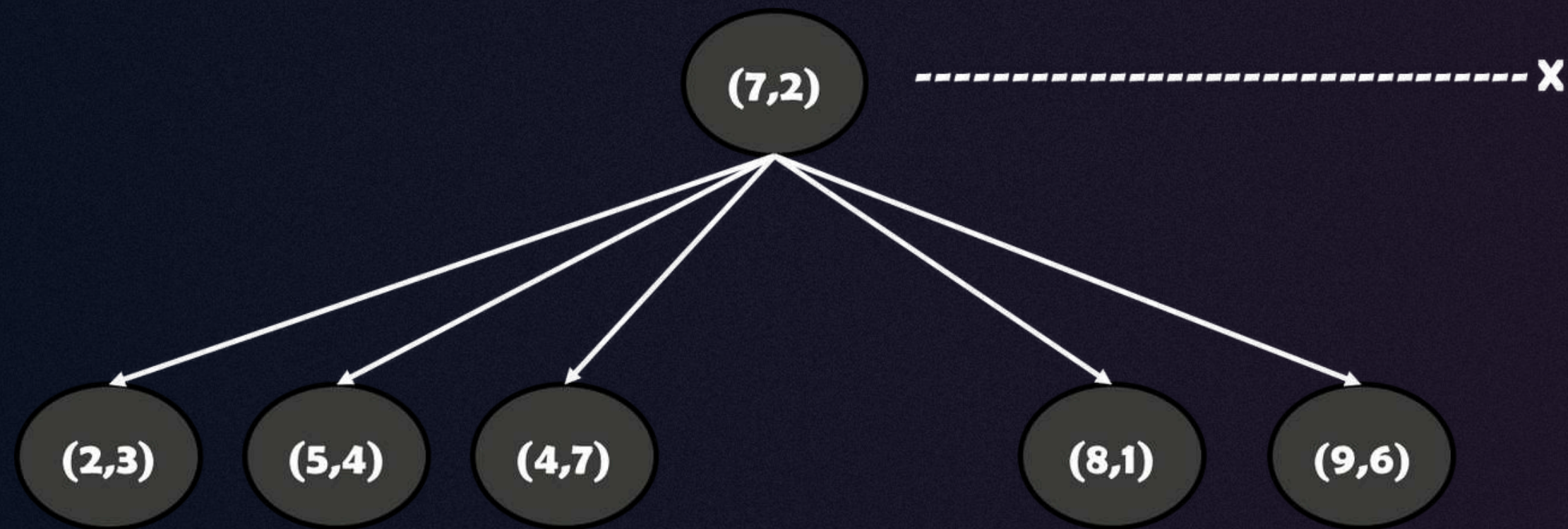
RESULTADO

01

Raíz = (7, 2)

Posibles hijos izq: (2,3), (5,4), (4,7)

Posibles hijos der: (8,1), (9,6)



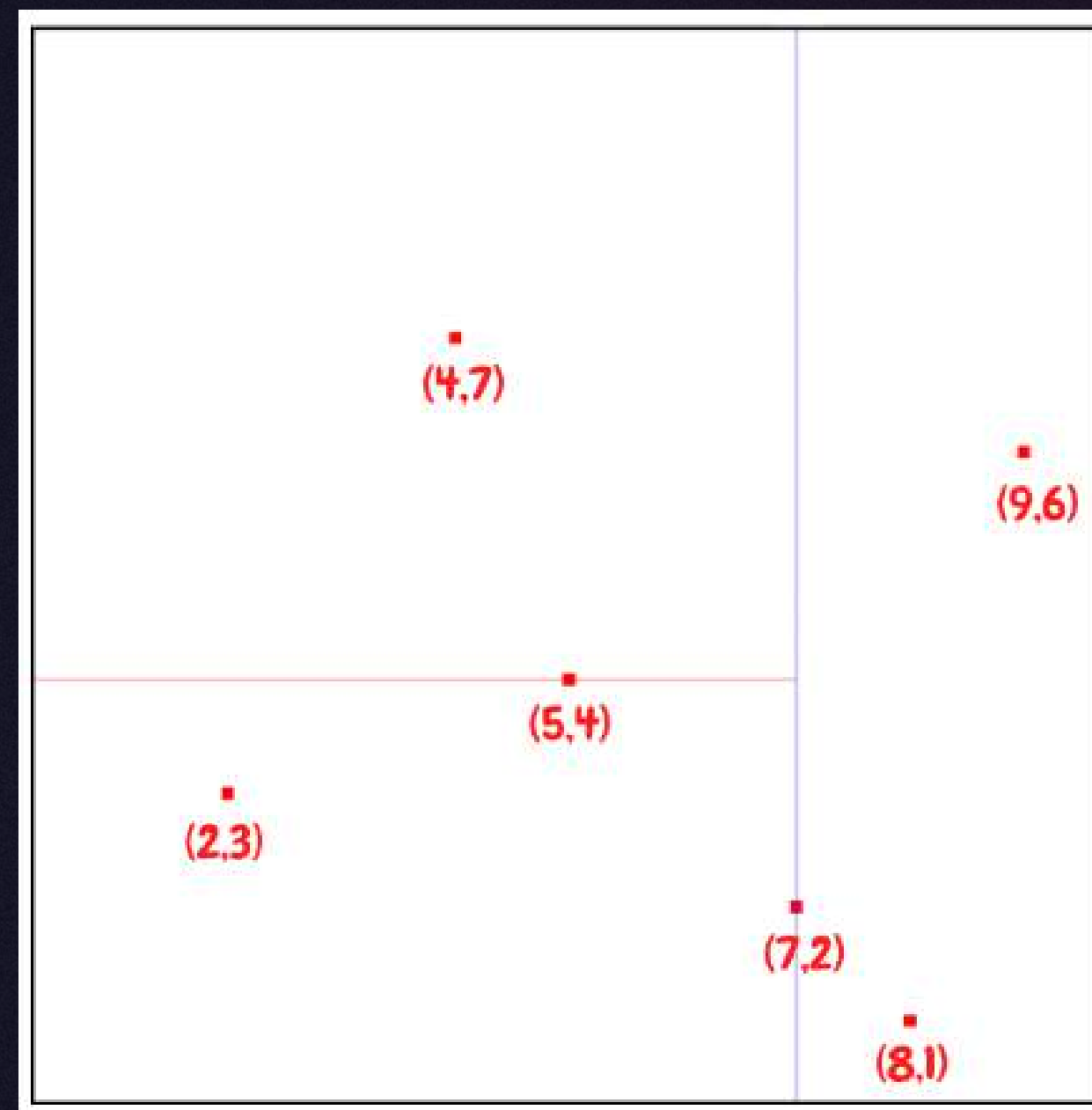
EJEMPLO EN 2D

(2,3), (5, 4), (4, 7)

Eje = 1 mod 2 = 1

PUNTOS ORDENADOS

(2, 3), (5, 4), (4, 7)

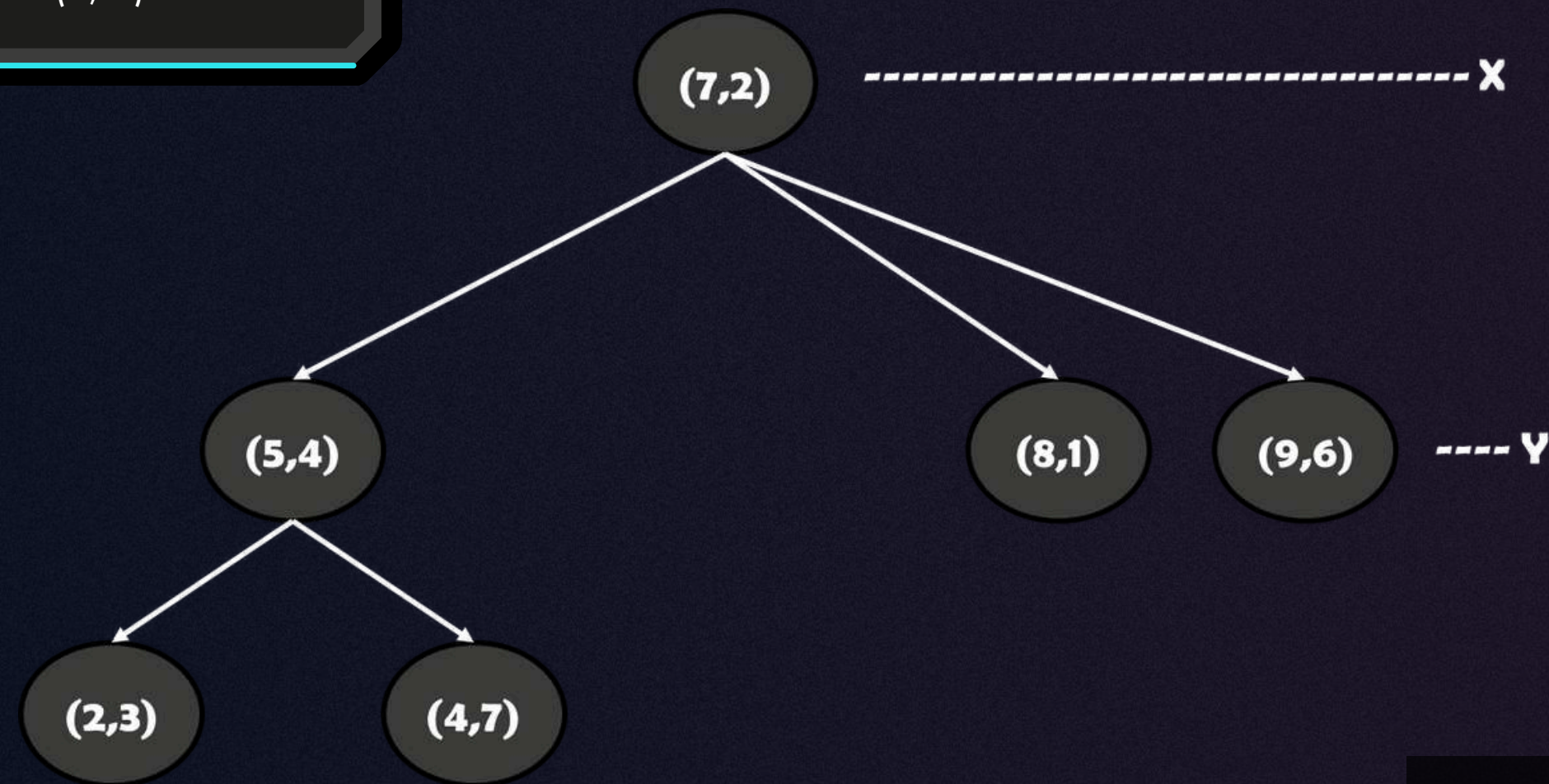


EJEMPLO EN 2D

RESULTADO

02

Nodo a Insertar = (5, 4)
Posibles hijos izq: (2,3)
Posibles hijos der: (4, 7)



EJEMPLO EN 2D

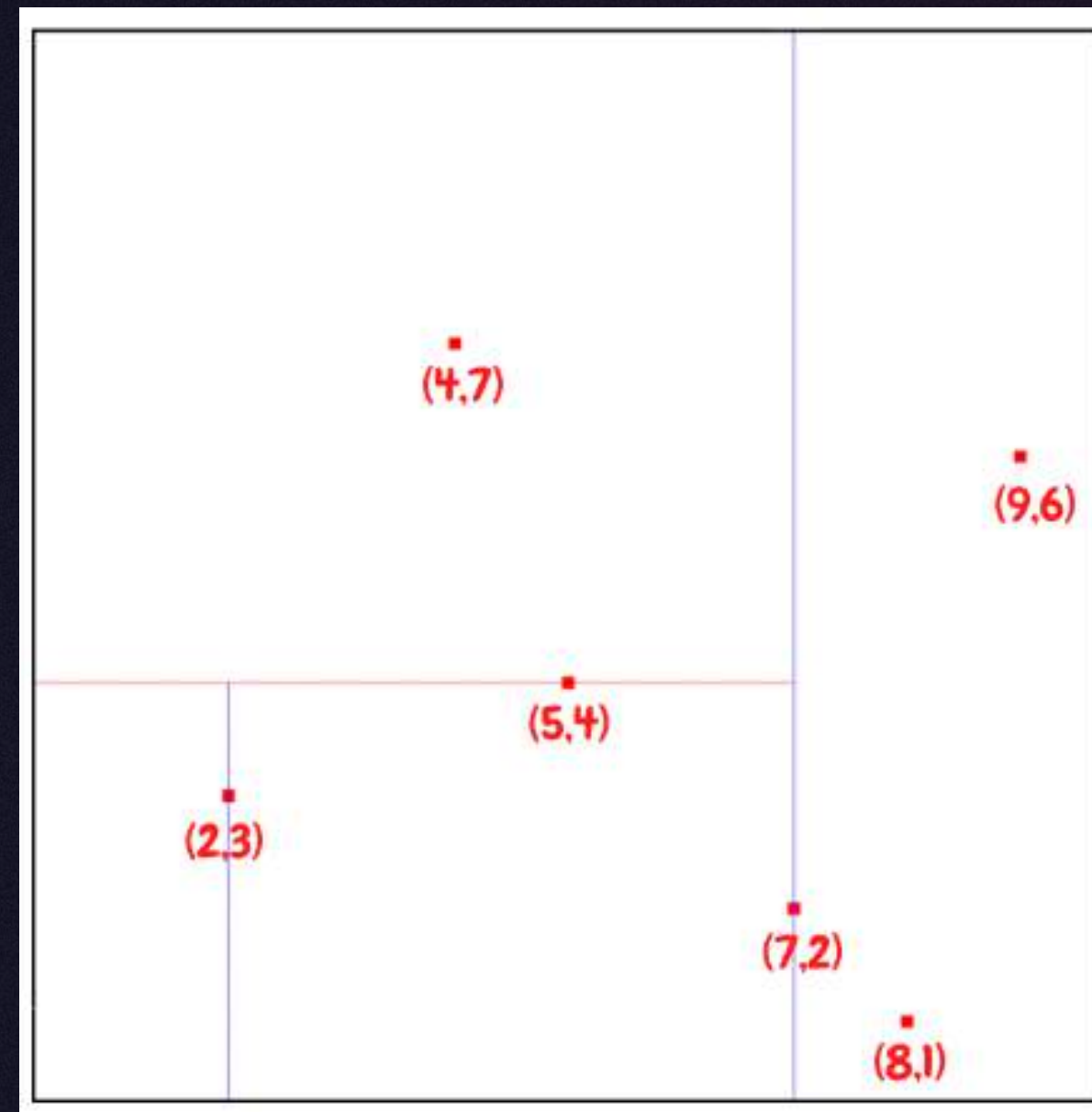
(2, 3)

×

Eje = 2 mod 2 = 0

PUNTOS ORDENADOS

(2, 3)

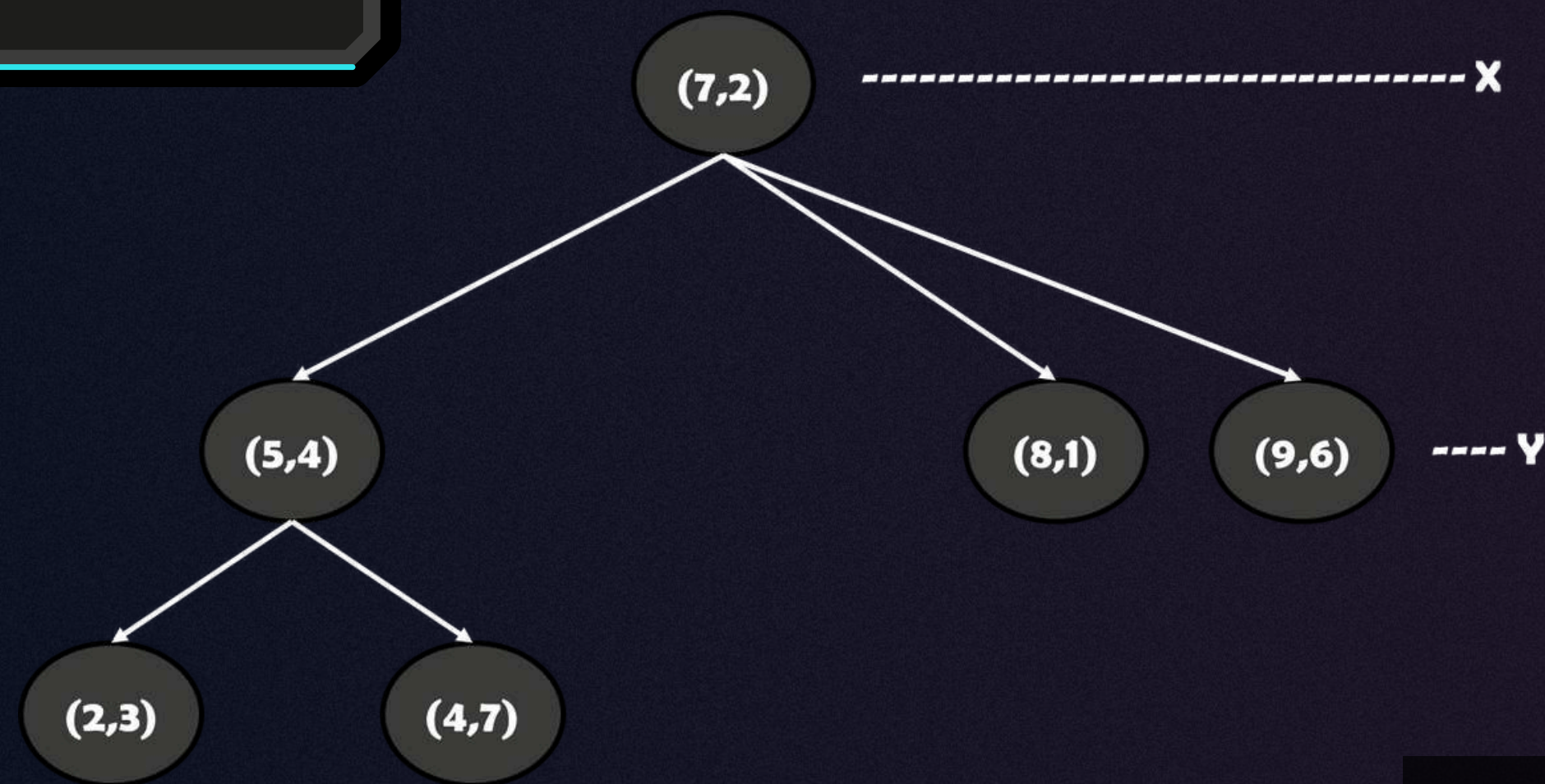


EJEMPLO EN 2D

RESULTADO

03

Nodo a Insertar = (2, 3)
Posibles hijos izq: null
Posibles hijos der: null



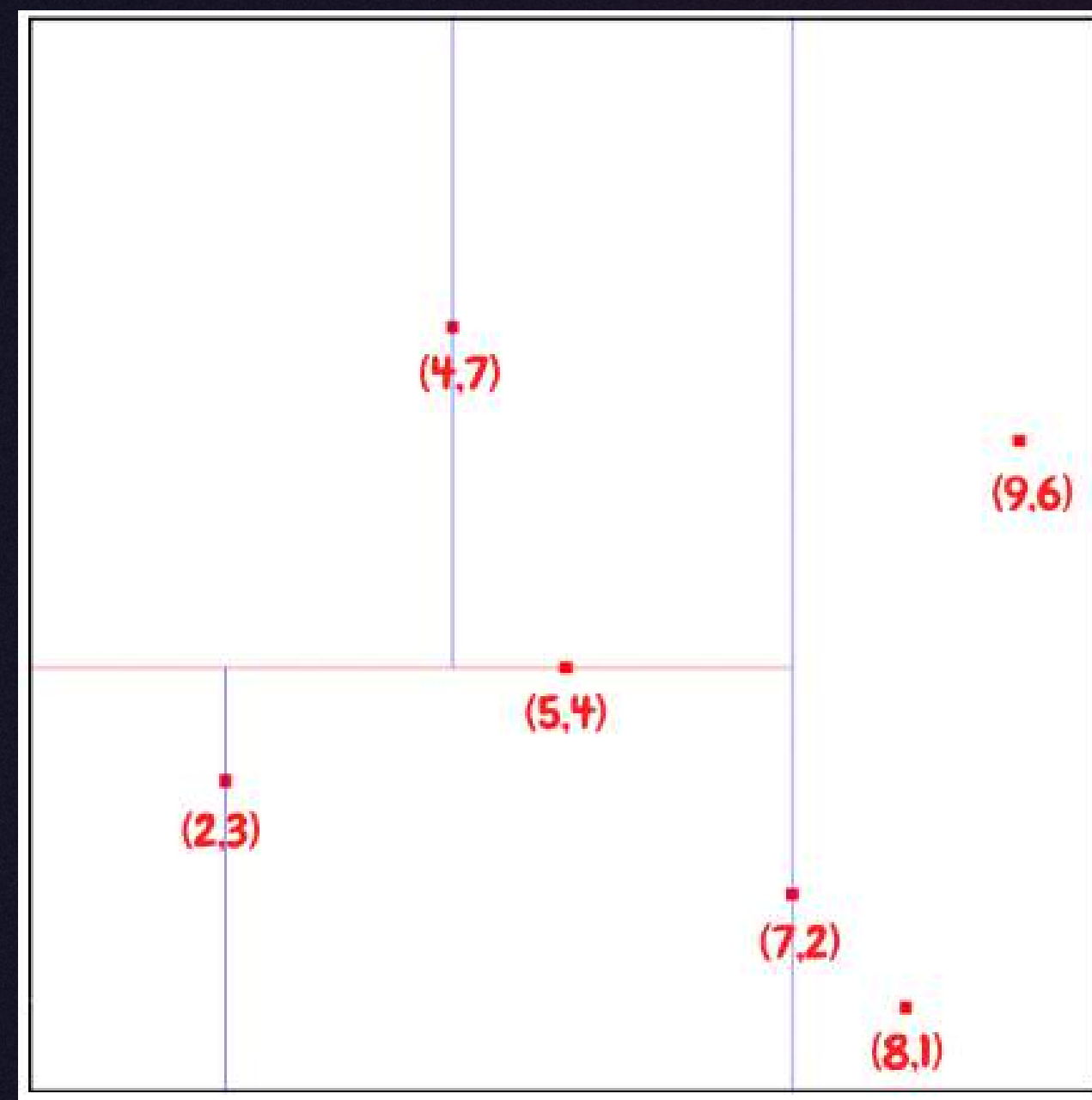
EJEMPLO EN 2D

(4, 7)

× Eje = 2 mod 2 = 0

PUNTOS ORDENADOS

(4, 7)

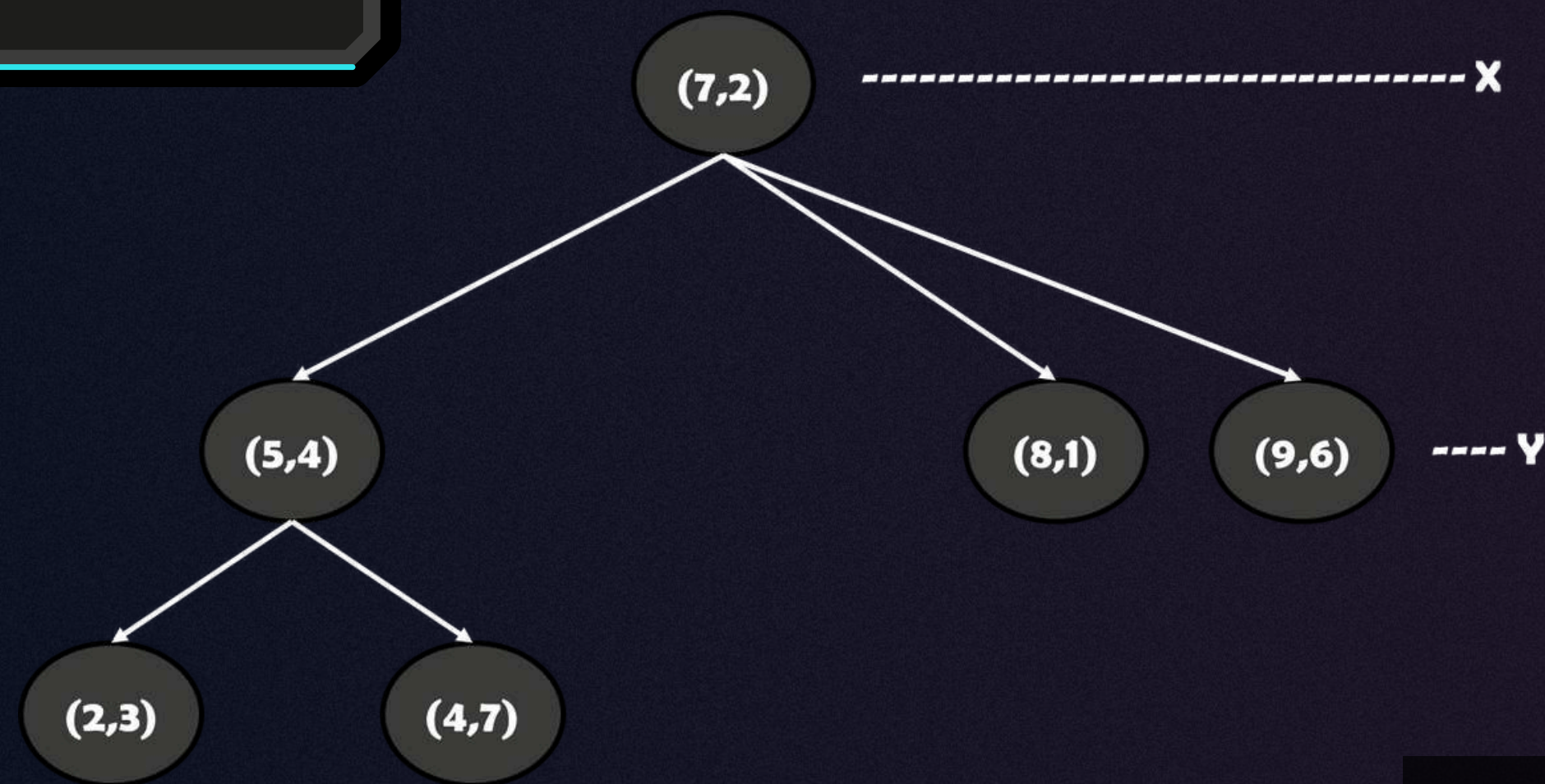


EJEMPLO EN 2D

RESULTADO

04

Nodo a Insertar = (4, 7)
Posibles hijos izq: null
Posibles hijos der: null



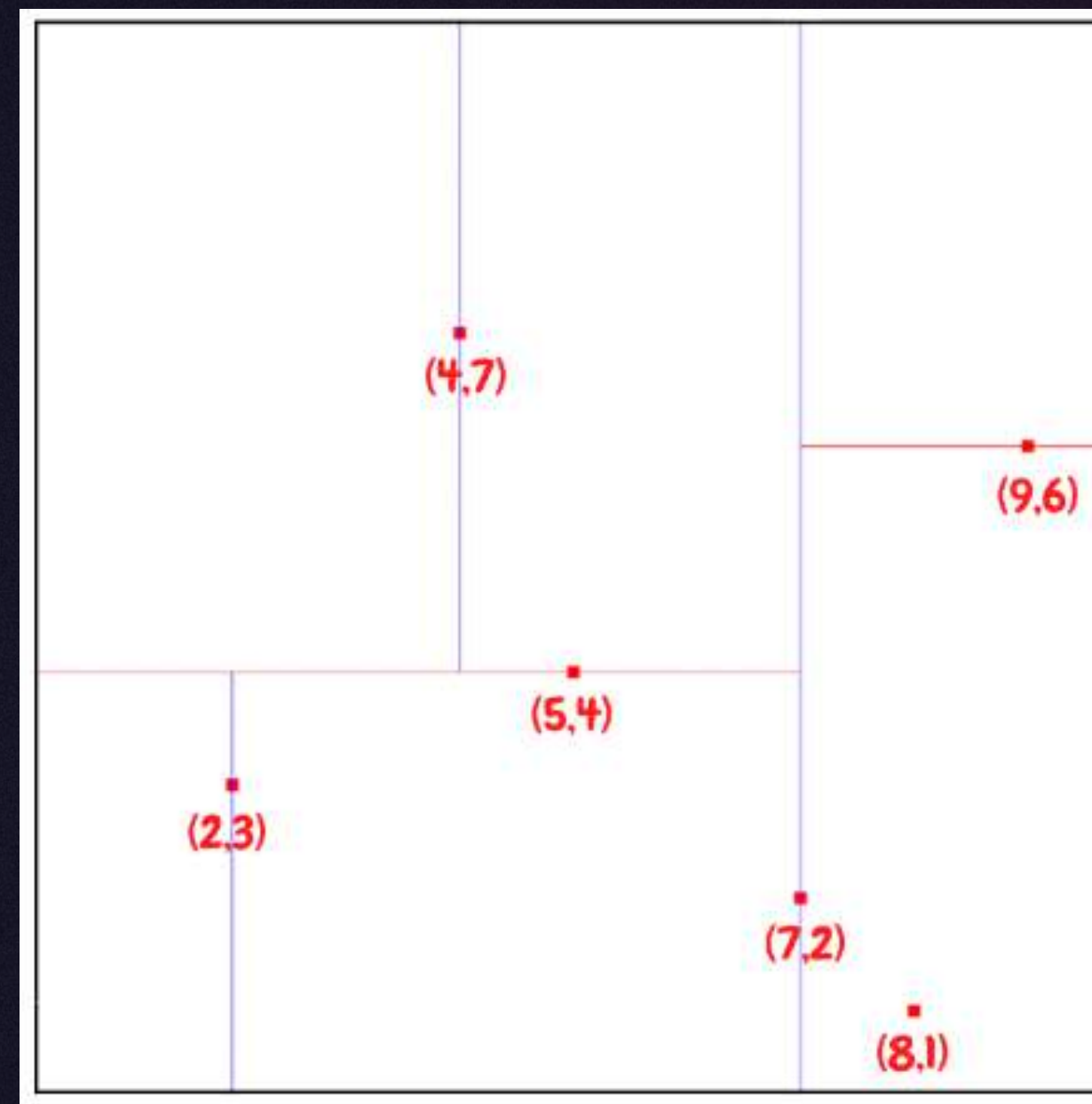
EJEMPLO EN 2D

(8,1), (9, 6)

Eje = $1 \bmod 2 = 1$

PUNTOS ORDENADOS

(8, 1), (9, 6)



EJEMPLO EN 2D

RESULTADO

05

Nodo a Insertar = (9, 6)
Posibles hijos izq: (8, 1)
Posibles hijos der: null



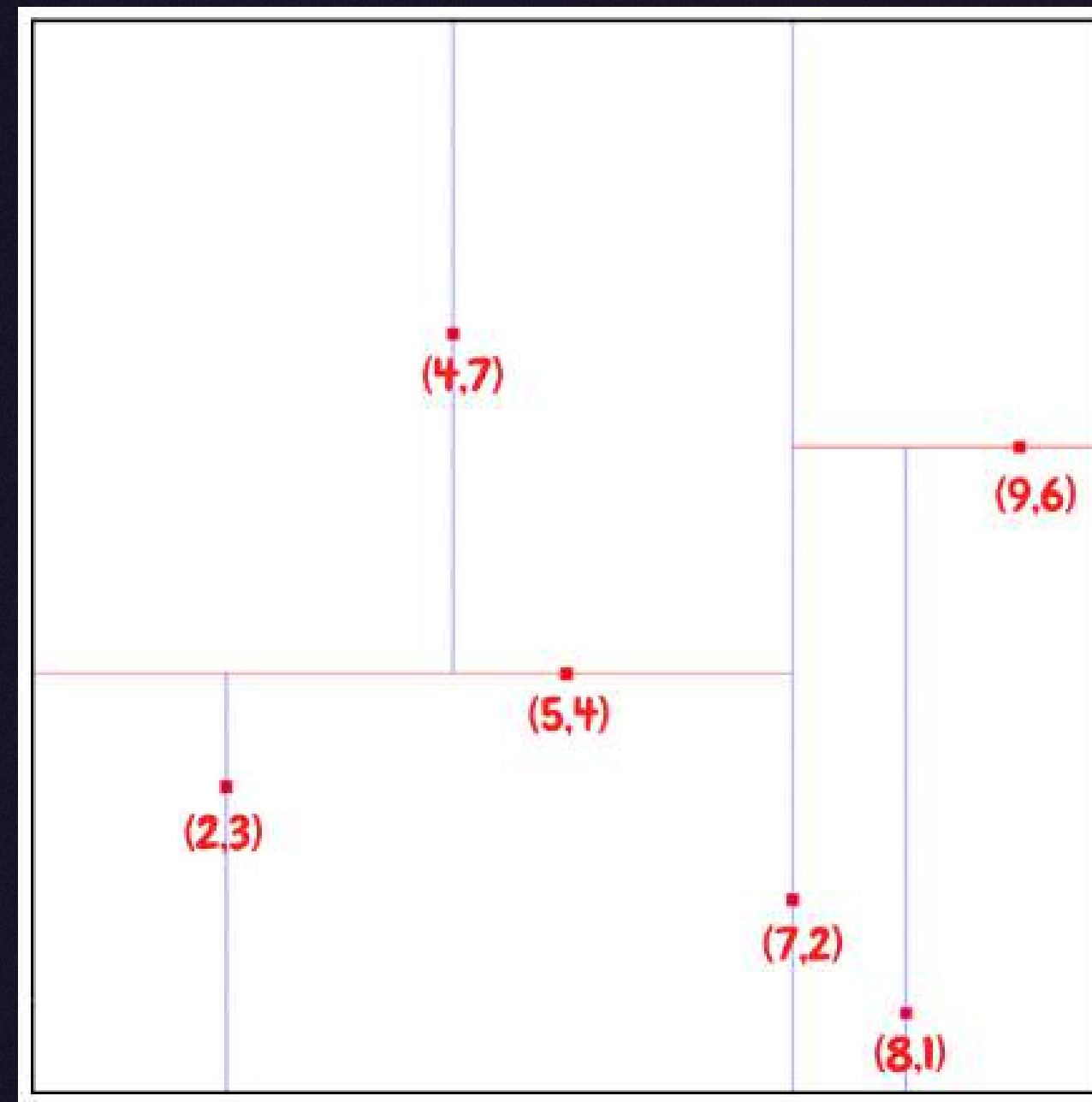
EJEMPLO EN 2D

(8, 1)

× Eje = $2 \bmod 2 = 0$

PUNTOS ORDENADOS

(8, 1)



EJEMPLO EN 2D

RESULTADO

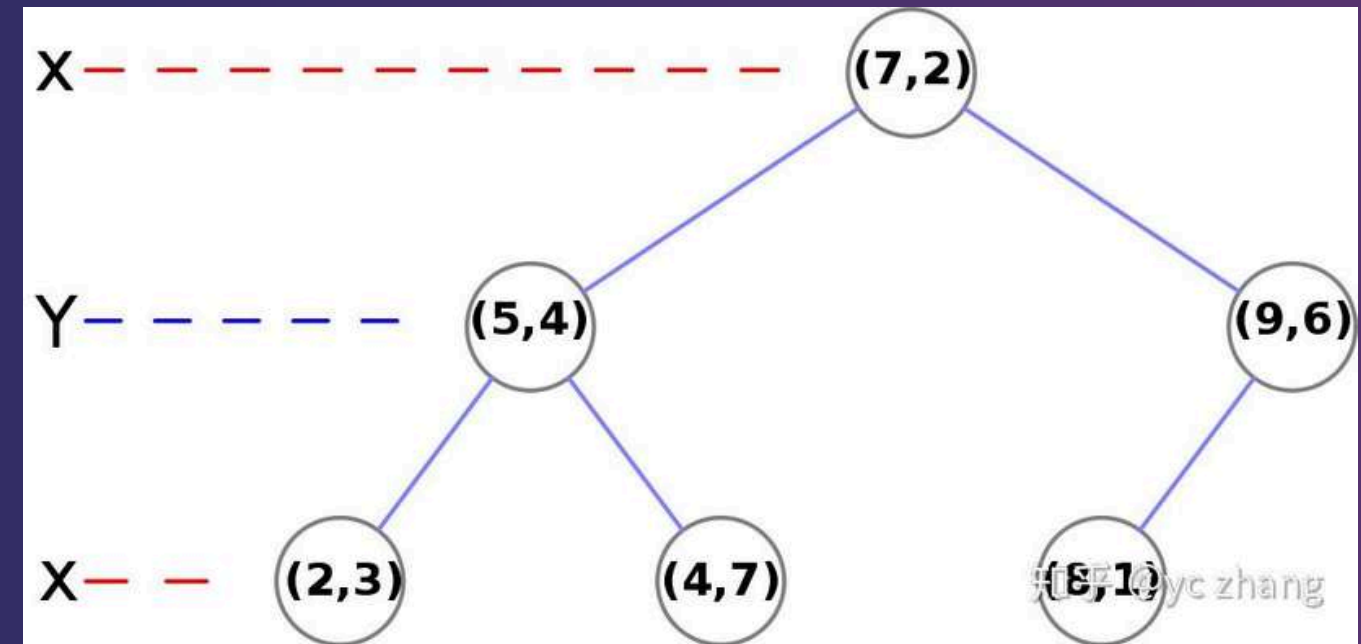
06

Nodo a Insertar = (8, 1)
Posibles hijos izq: null
Posibles hijos der: null



EJEMPLO EN 2D

BÚSQUEDA Y APLICACIONES





Búsqueda en Kd-Tree

Dos tipos de búsqueda

Nearest Neighbor

¿Cuál es el punto más cercano a Q?

Range Search

¿Qué puntos están dentro del rango R?

Estructura en memoria



Algoritmo Nearest Neighbor

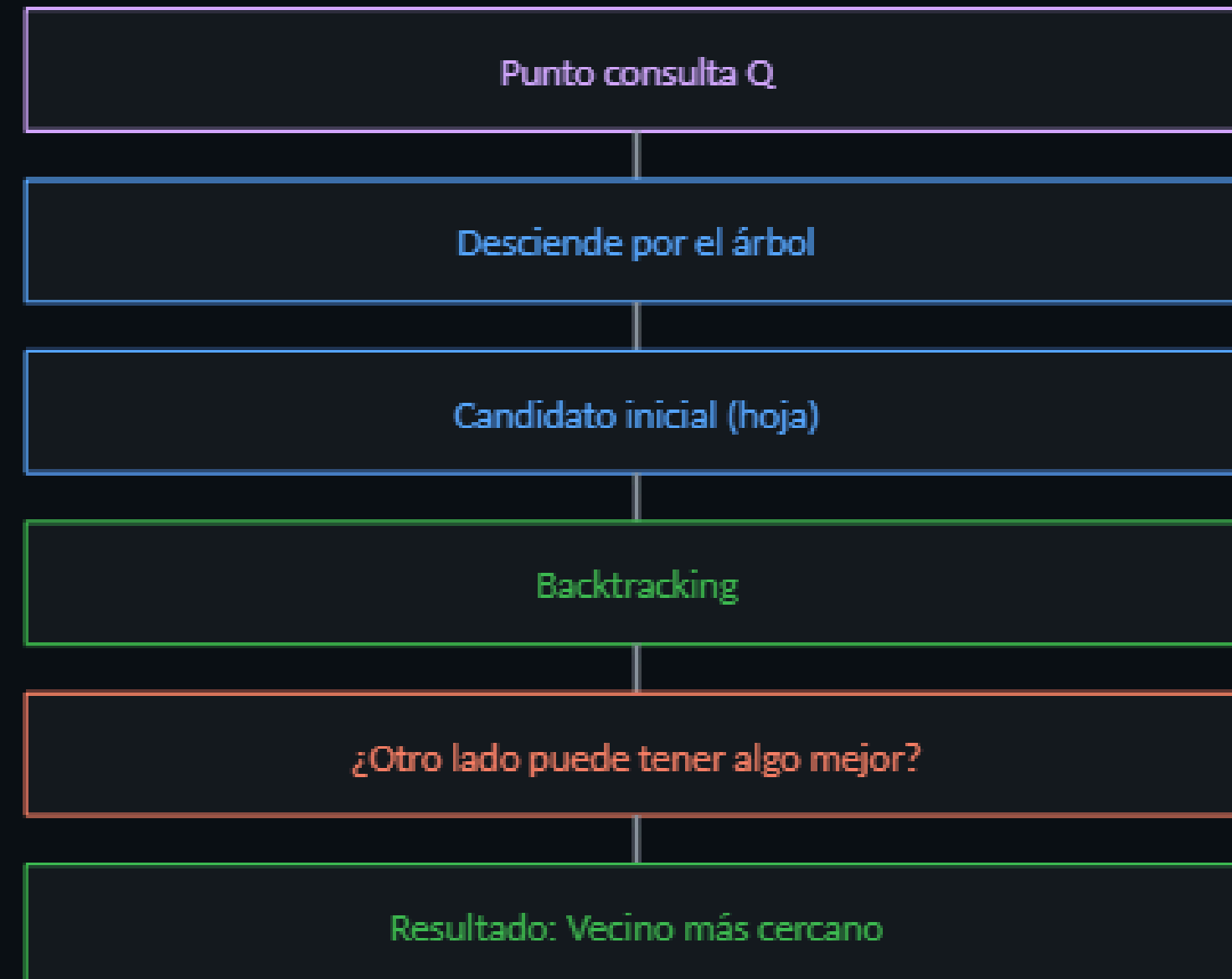
FASE 1 — Descenso

- Baja desde la raíz siguiendo la lógica de cada dimensión.
- En cada nodo: ¿Q está a la izquierda o derecha del corte?
- Al llegar a una hoja → candidato inicial encontrado.

FASE 2 — Backtracking

- Sube revisando si el otro lado del corte puede tener algo mejor.
- Usa pruning para descartar ramas completas.
- Actualiza el mejor candidato si encuentra uno más cercano.

Flujo de ejecución



Poda (Pruning)

La razón por la que el KD-Tree es eficiente

Regla del pruning:

Si la distancia de Q al plano de corte es MAYOR que la distancia al mejor candidato actual → esa rama COMPLETA se descarta.

NO podar — explorar

El círculo centrado en Q cruza el plano de corte. Podría haber un punto más cercano al otro lado. Hay que explorar esa rama.

PODAR — descartar

El círculo centrado en Q NO cruza el plano. Ningún punto al otro lado puede estar más cerca que el candidato actual. Se descarta toda la rama.

Complejidad de Búsqueda

Caso promedio

$$O(\log N)$$

Con pocas dimensiones y árbol balanceado

Peor caso

$$O(N)$$

Árbol desbalanceado o muchas dimensiones

Fuerza bruta

$$O(N)$$

Siempre, sin importar nada

Búsqueda por Rango

Complejidad: $O(\sqrt{N} + k)$ en 2D, donde k es la cantidad de puntos encontrados.

En dimensiones más altas, sube. El pruning sigue ayudando: si el plano no intersecta el rango, se descarta esa rama.

Búsqueda por Rango

Encontrar TODOS los puntos dentro de una región

⚠ Diferencia clave vs Nearest Neighbor

No paras cuando encuentras uno. Sigues explorando hasta recorrer todas las ramas relevantes.

Algoritmo paso a paso:

1. En cada nodo: ¿está dentro del rango? → agrega a resultados.
2. ¿El plano de corte intersecta el rango? → explora ese subárbol.
3. Si no intersecta → poda, descarta esa rama completa.
4. Continúa hasta revisar todas las ramas relevantes.

Ejemplo visual



Aplicaciones del KD-Tree

Machine Learning

K-Nearest Neighbors

scikit-learn usa KD-Trees internamente para KNN cuando las dimensiones son bajas. Cada predicción pasa de $O(N)$ a $O(\log N)$.

Videojuegos

Detección de colisiones · Ray casting

Colisiones en tiempo real, inteligencia artificial de NPCs, ray tracing en motores gráficos 3D.

Sistemas GIS

Geografía e información espacial

Hospital más cercano, puntos dentro de una zona, análisis geoespacial. PostGIS usa estructuras similares.

Machine Learning — KNN

¿Qué hace KNN?

Para clasificar un punto nuevo, busca sus K vecinos más cercanos en los datos de entrenamiento y decide por votación.

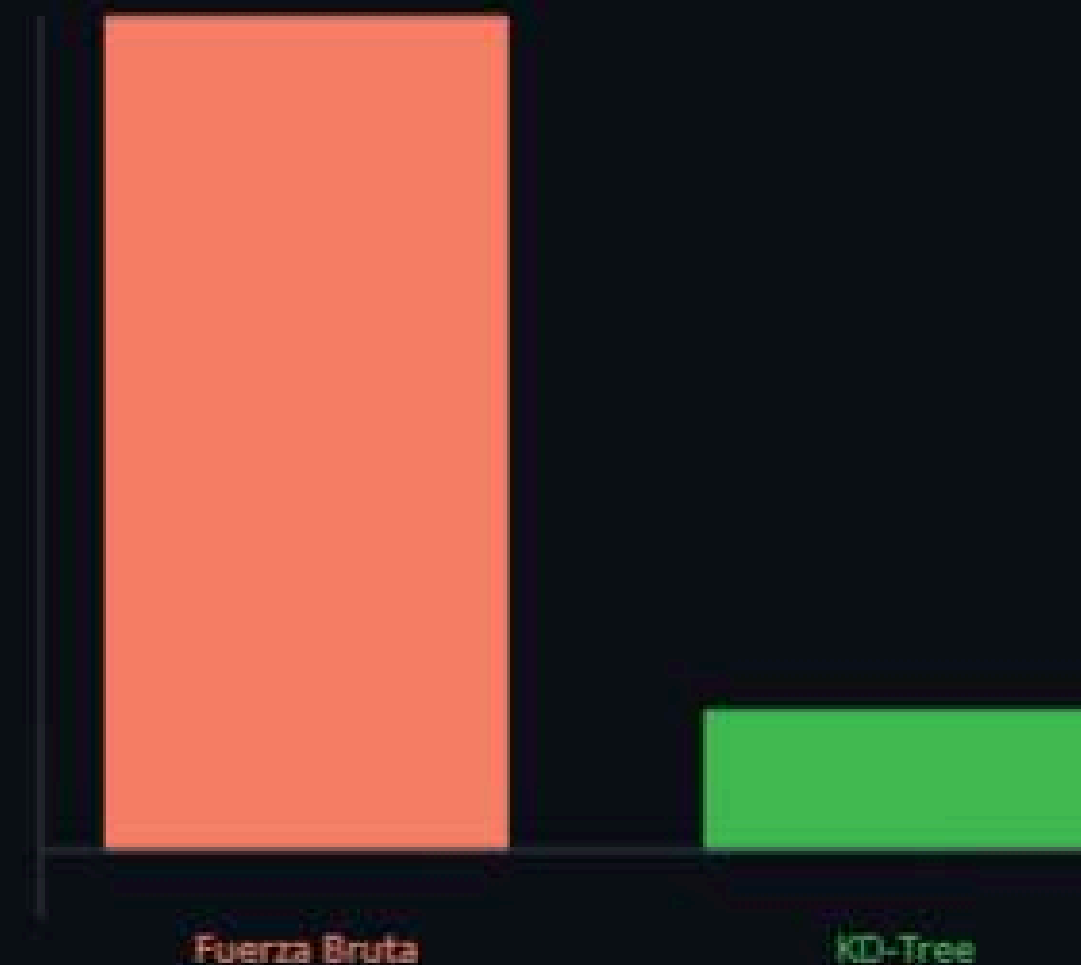
El problema sin KD-Tree

Con millones de datos, encontrar los K vecinos por fuerza bruta cuesta $O(N)$ por cada predicción. Inaceptable en producción.

Con KD-Tree

Se construye el árbol UNA vez. Cada predicción se resuelve en $O(\log N)$. La diferencia es la que hace el sistema usable.

Comparación de tiempo



📖 scikit-learn usa KD-Trees internamente en `KNeighborsClassifier` para datos de baja dimensión.

Videojuegos y Sistemas GIS

Videojuegos y Gráficos

Detección de colisiones

Solo revisa objetos cercanos al personaje, no toda la escena.

Ray casting / Ray tracing

Encuentra qué geometría intersecta cada rayo sin revisar todos los triángulos.

IA de NPCs

El enemigo sabe qué aliados o enemigos tiene cerca en tiempo real.

Sistemas GIS

Hospital más cercano

Nearest Neighbor con millones de puntos geográficos.

Puntos dentro de zona

Range search: estaciones de recarga dentro de N km de tu ruta.

Análisis geoespacial

PostGIS y otras herramientas GIS usan estructuras tipo KD-Tree.

Limitaciones del KD-Tree

🧠 Curse of Dimensionality (Maldición de la dimensionalidad)

El KD-Tree brilla en 2D y 3D. Funciona bien hasta ~20 dimensiones. Pero cuando hablamos de 50, 100 o 500 dimensiones (como en ML moderno), **se vuelve tan lento como la fuerza bruta**.

¿Por qué?

En altas dimensiones, todos los puntos tienden a estar aproximadamente a la misma distancia de Q. Las distancias se 'aplanan'.

Efecto en el pruning

La esfera de búsqueda en Q tiene que crecer mucho para capturar algún punto, cruzando casi todos los planos de corte. El pruning deja de funcionar.

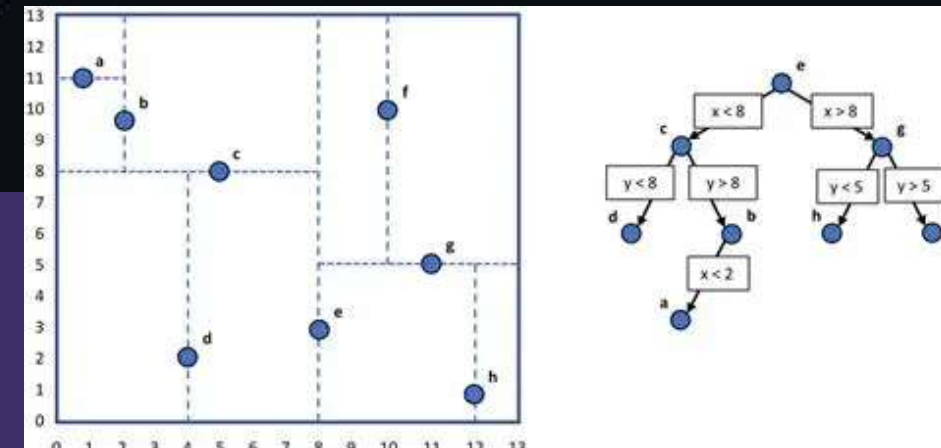
Resultado práctico

El KD-Tree termina visitando casi todos los nodos. La ventaja sobre $O(N)$ desaparece casi por completo.

Otras limitaciones:

➤ Datos dinámicos: inserciones frecuentes desbalancean el árbol, obligando a reconstruirlo.

📦 Alta dimensión: almacenar el árbol también se vuelve costoso en memoria.



Comparación de Eficiencia

Situación	Fuerza Bruta	KD-Tree
Construcción	$O(1)$ — no construye	$O(N \log N)$
Búsqueda NN — pocas dims	$O(N)$	$O(\log N)$ promedio $\bar{u}$
Búsqueda NN — muchas dims	$O(N)$	$O(N)$ peor caso Δ
Actualización dinámica	$O(1)$ por inserción	Costoso — rebalanceo $\bar{u}$
Uso de memoria	$O(N)$	$O(N)$

Conclusiones

1

El KD-Tree no solo se construye — se usa. La búsqueda es donde se justifica todo el esfuerzo de construcción.

2

El backtracking con pruning es el mecanismo clave: permite descartar ramas sin explorarlas, pasando de $O(N)$ a $O(\log N)$.

3

Tiene aplicaciones reales y relevantes: KNN en ML, colisiones en videojuegos, análisis geoespacial en GIS.

4

Tiene un límite claro: la maldición de la dimensionalidad. En alta dimensión se usan Ball Trees, LSH o búsqueda aproximada.

5

Saber cuándo usar y cuándo NO usar una estructura es tan importante como entender cómo funciona.

Active Windows

REFERENCIAS

- <https://datasciencepythonblog.net/arboles-k-dimensionales-scipy/>
- <https://medium.com/@polkamnithyasri/kd-tree-and-ball-tree-algorithms-aa99dfad4ceb>
- <https://es.statisticseasily.com/glossario/what-is-kd-tree-a-comprehensive-guide/>
- https://www.tutorialspoint.com/data_structures_algorithms/k_d_trees.htm
- <https://medium.com/@isurangawarnasooriya/exploring-kd-trees-a-comprehensive-guide-to-implementation-and-applications-in-python-3385fd56a246#941a>
- <https://courses.grainger.illinois.edu/cs225/fa2019/resources/kd-tree/>
- <https://cs186berkeley.net/notes/note19/>
- <https://www.geeksforgeeks.org/cpp/kd-trees-in-cpp/>
- <https://www.cs.cmu.edu/~ckingsf/bioinfo-lectures/kdtrees.pdf>
- <https://www.youtube.com/watch?v=XG4zpiJkD4>
- https://en.wikipedia.org/wiki/K-d_tree
- <https://codeofcode.org/lessons/k-dimensional-k-d-trees/>
- <https://arxiv.org/pdf/2112.06188>
- <https://yashen.github.io/post/kd-tree/>
- <https://www.youtube.com/watch?v=X46vqqutpBA>
- <https://users.dcc.uchile.cl/~gnavarro/ps/cita98.pdf>
- <https://datasciencepythonblog.net/kd-tree-scikit-learn/>
- <https://www.geeksforgeeks.org/dsa/search-and-insertion-in-k-dimensional-tree/>
- https://www.youtube.com/watch?v=_mSVK00aaYw
- <https://programmerclic.com/article/8896910634/>
- <https://www.geeksforgeeks.org/cpp/kd-trees-in-cpp/>

¡ GRACIAS !